

Linked List

Linked List

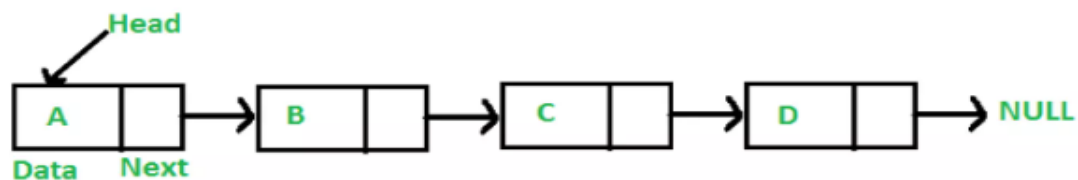
Linked List is basically **chains of nodes** where each node contains information such as **data** and a **pointer to the next node** in the chain. It is a popular data structure with a wide range of real-world applications. Unlike Arrays, Linked List elements are not stored at a contiguous location. In the linked list there is a **head pointer**, which points to the first element of the linked list, and if the list is empty then it simply points to null or nothing.

Basic Terminologies of Linked List

- **Head:** The Head of a linked list is a pointer to the first node or reference of the first node of linked list. This pointer marks the beginning of the linked list.
- **Node:** Linked List consists of a series of nodes where each node has two parts: **data** and **next pointer**.
- **Data:** Data is the part of node which stores the information in the linked list.
- **Next pointer:** Next pointer is the part of the node which points to the next node of the linked list.

Linked list

- linear data structure
- Elements are not stored at contiguous memory locations
- Elements in a linked list are linked using pointers



TYPES

- Linked list consists of nodes where each node contains a data field and a reference(link) to the next node in the list.
 - Linked list comprise of group or list of nodes in which each node have link to next node to form a chain
 - Types of linked list
 - Singly linked list
 - Doubly linked list
 - Circular linked list
-

ARRANGE THE NAME OF THE STUDENTS

Arrange the name of the students according to the first letter of their names.



MICHAEL

JOHN

ALPHA

MARK

SMITH

TONY



ARRANGE THE NAME OF THE STUDENTS

Arrange the name of the students according to the first letter of their names.

ALPHA	JOHN	MARK	MICHAEL	ROCK	SMITH	TONY
-------	------	------	---------	------	-------	------

WAYS TO MAINTAIN A LIST IN MEMORY

Two Ways

ARRAYS



ARRAY

LINKED LIST

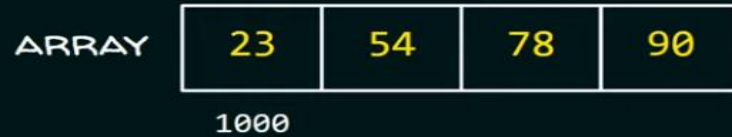


LINKED LIST

Why Linked List?

- **Arrays have the following limitations**
 - The size of the array is fixed
 - To insert a new element in an array, existing elements have to be shifted
- **Advantages of LL over arrays**
 - Dynamic size
 - Ease of insertion/deletion
 - Random access is not allowed

We want to store a list of numbers: 23, 54, 78, 90



ASSUMING SIZE OF
INT IS 4 BYTES.

We want to store a list of numbers: 23, 54, 78, 90



In an array, elements are stored in consecutive memory locations.

ARRAY IS THE SEQUENTIAL REPRESENTATION OF LIST.

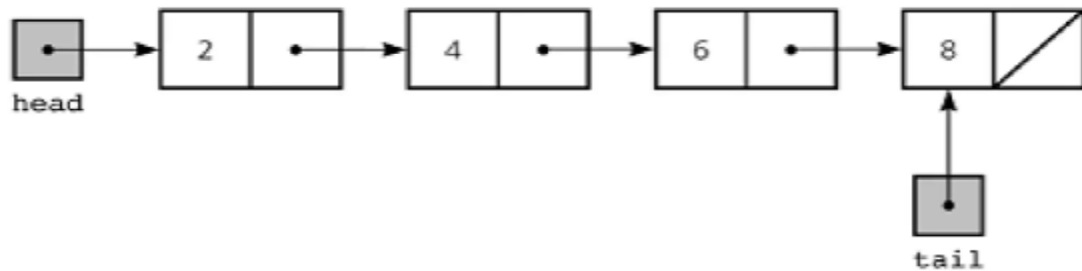
Linked List vs Array

Array	Linked List
data structure that contains a collection of similar type of data elements	non-primitive data structure contains a collection of unordered linked elements known as nodes
Accessing an element in an array is fast	Accessing an element in an array is bit slower.
Operations in arrays consume a lot of time	operations in Linked lists is fast
Arrays are of fixed size.	Linked lists are dynamic and flexible and can expand and contract its size.
In array, memory is assigned during compile time	Linked list it is allocated during execution or runtime.
Elements are stored consecutively in arrays	Elements are stored randomly in Linked lists.
requirement of memory is less due to actual data being stored within the index in the array	more memory in Linked Lists due to storage of additional next and previous referencing elements.
memory utilization is inefficient in the array	memory utilization is efficient in the linked list.

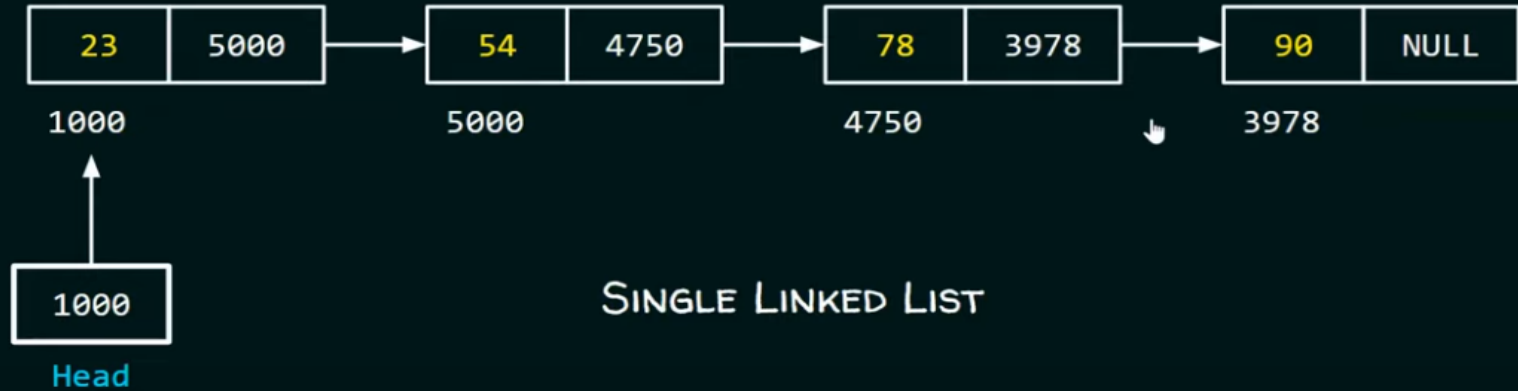
Representation of linked list

- Represented by a pointer to the first node of the linked list
- The first node is called the head.
- If the linked list is empty, then the value of the head is NULL.
- Each node in a list consists of at least two parts:
 - 1) data
 - 2) Pointer (Or Reference) to the next node

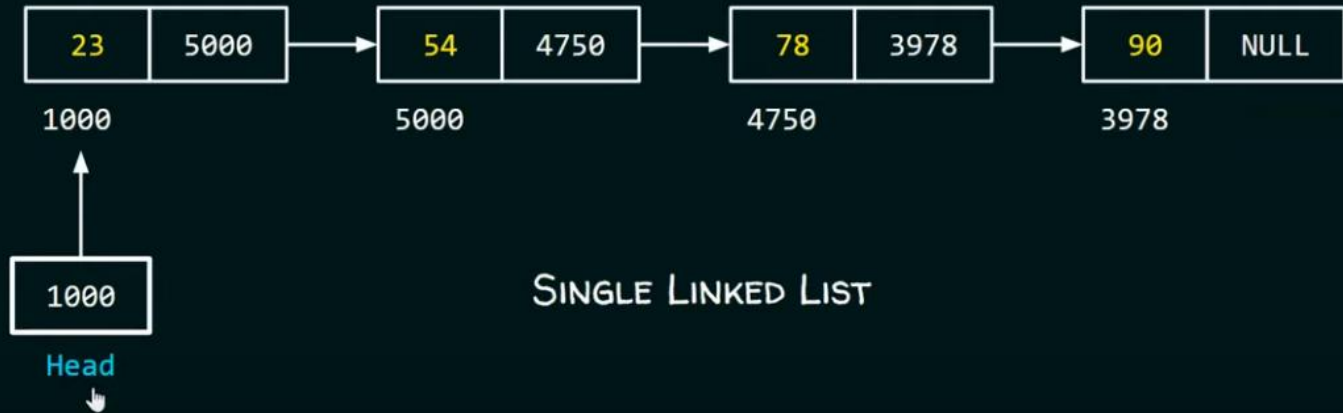
Representation of linked list



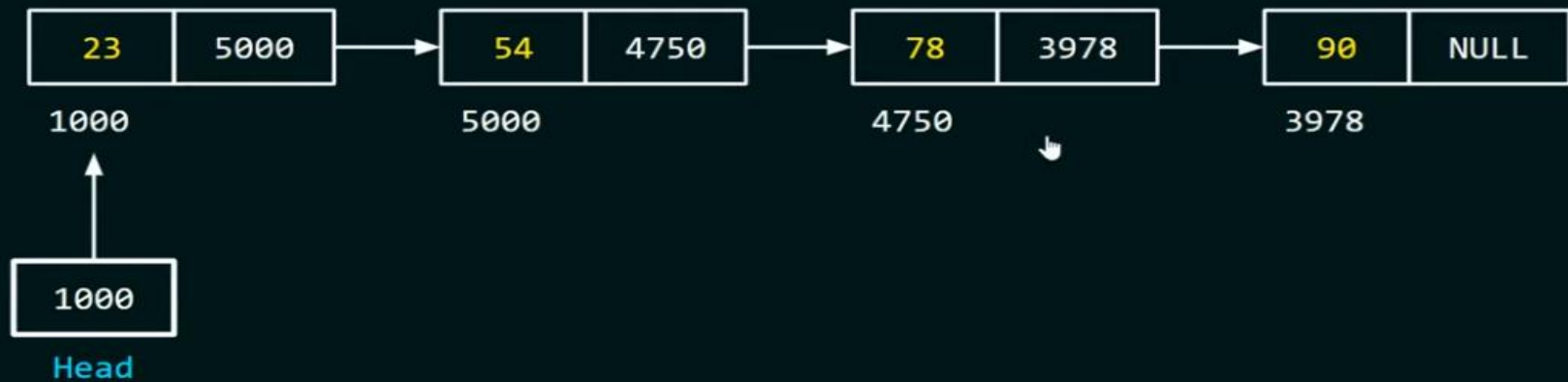
We want to store a list of numbers: 23, 54, 78, 90



We want to store a list of numbers: 23, 54, 78, 90



SINGLE LINKED LIST



Nodes are scattered here and there in the memory but they are still connected with each other.

Basic Operations

- **Insertion** – Adds an element into the list
- **Deletion** – Deletes an element from the list
- **Display** – Displays the complete list
- **Search** – Searches an element using the given key

TYPES OF LINKED LIST



SINGLE LINKED LIST: Navigation is forward only.



DOUBLY LINKED LIST: Forward and backward navigation is possible.



CIRCULAR LINKED LIST: Last element is linked to the first element. 🖱️

WHAT IS A SINGLE LINKED LIST?



SINGLE LINKED LIST

A single linked list is a list made up of nodes that consists of two parts.



Data



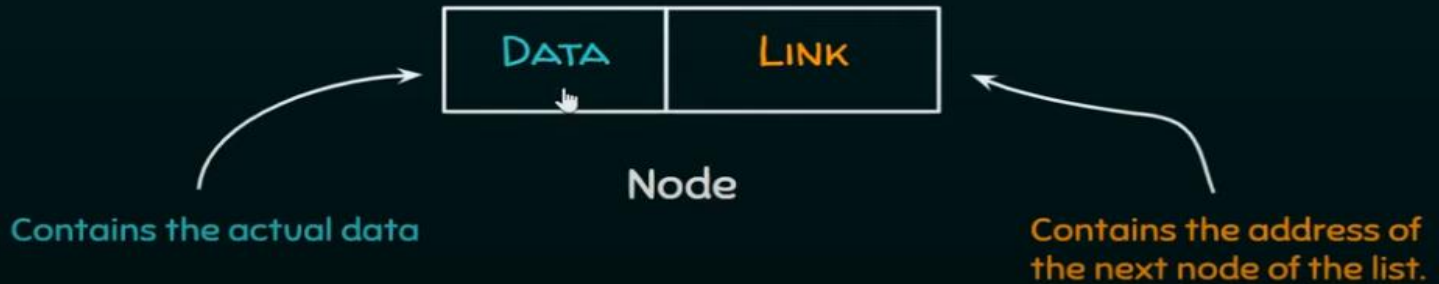
Link

SINGLE LINKED LIST

A single linked list is a list made up of nodes that consists of two parts.

★ Data

★ Link



REPRESENTATION OF A SINGLE LINKED LIST

Suppose, we want to store a list of numbers: 23, 54, 78, 90



1000



2000



3000



4000

REPRESENTATION OF A SINGLE LINKED LIST

Suppose, we want to store a list of numbers: 23, 54, 78, 90



1000



2000



3000



4000

THE SOLUTION



How to Create Node ?????

Self Referential Structure is a structure which contains a pointer to a structure of the same type.

```
struct abc {  
    int a;  
    char b;  
    struct abc *self;  
};
```



NODE REPRESENTATION IN C

```
struct node {  
    int data;  
    struct node *link;  
};
```



Node

CREATING A NODE IN C

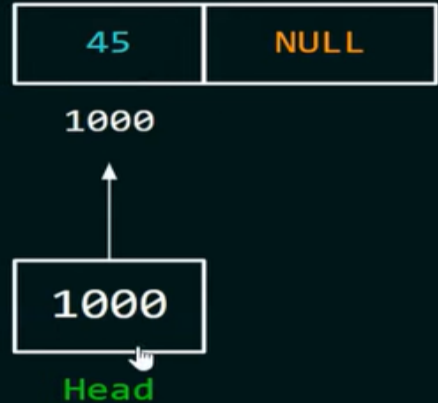
```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *link;
};

int main() {
    struct node *head = NULL;
    head = (struct node *)malloc(sizeof(struct node));

    head->data = 45;
    head->link = NULL;

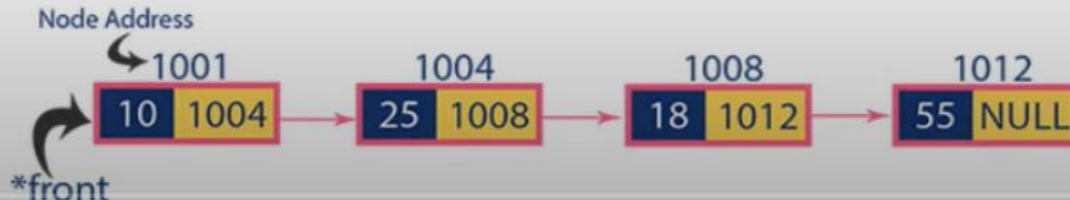
    printf("%d", head->data);
    return 0;
}
```



[w.programiz.com/online-compiler/1jRhwsrDcv8Xz](https://www.programiz.com/online-compiler/1jRhwsrDcv8Xz)

What is Singly Linked List?

- Single linked list is a sequence of elements in which every element has link to its next element in the sequence.
- In any single linked list, the individual element is called as **"Node"**.
- Every "Node" contains two fields, **Data field, and the Next field**.
- **Data field** is used to store actual value of the node.
- **Next field** is used to store the address of next node in the sequence.
- In a single linked list, the address of the first node is always stored in a reference node known as **"front" or "head"**.
- Always Next part (reference part) of the last node must be NULL.



Operations on Singly Linked List

1. Create

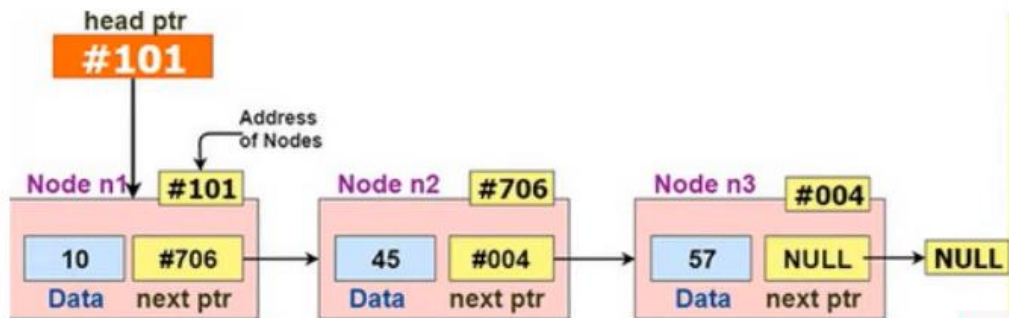
2. Insertion

- Inserting At Beginning of the list
- Inserting At End of the list
- Inserting At Specific location in the list

3. Deletion

- Deleting from Beginning of the list
- Deleting from End of the list
- Deleting a Specific Node

4. Display



Inserting Node in Linked List

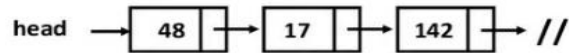
Case 1: Inserting At Beginning of the list

Step 1 - Create a **newNode** with given value.

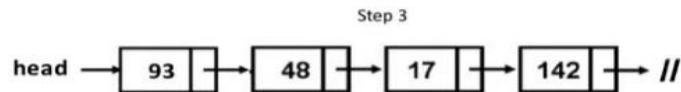
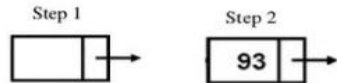
Step 2 - Check whether list is **Empty** (**head == NULL**)

Step 3 - If it is **Empty** then, set **newNode**→**next** = **NULL** and **head** = **newNode**.

Step 4 - If it is **Not Empty** then, set **newNode**→**next** = **head** and **head** = **newNode**.

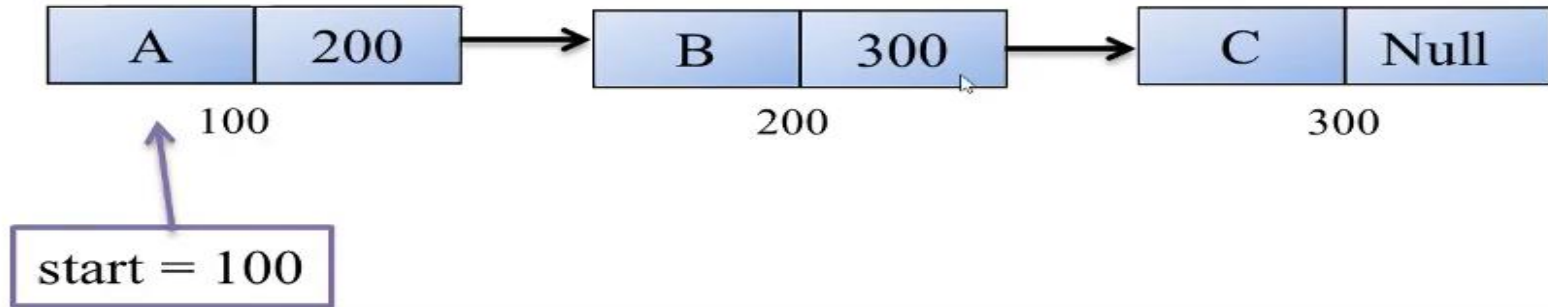


- Follow the previous steps and we get

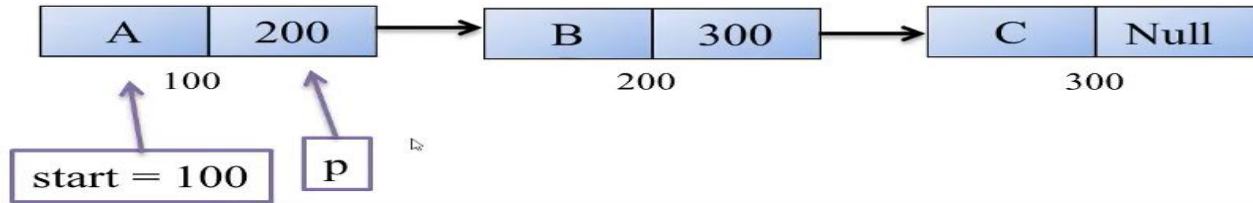


Insert a Node at particular Position

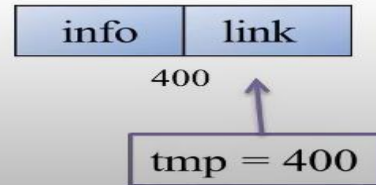
Singly Linked List Insertion at nth Position



Singly Linked List Insertion at nth Position

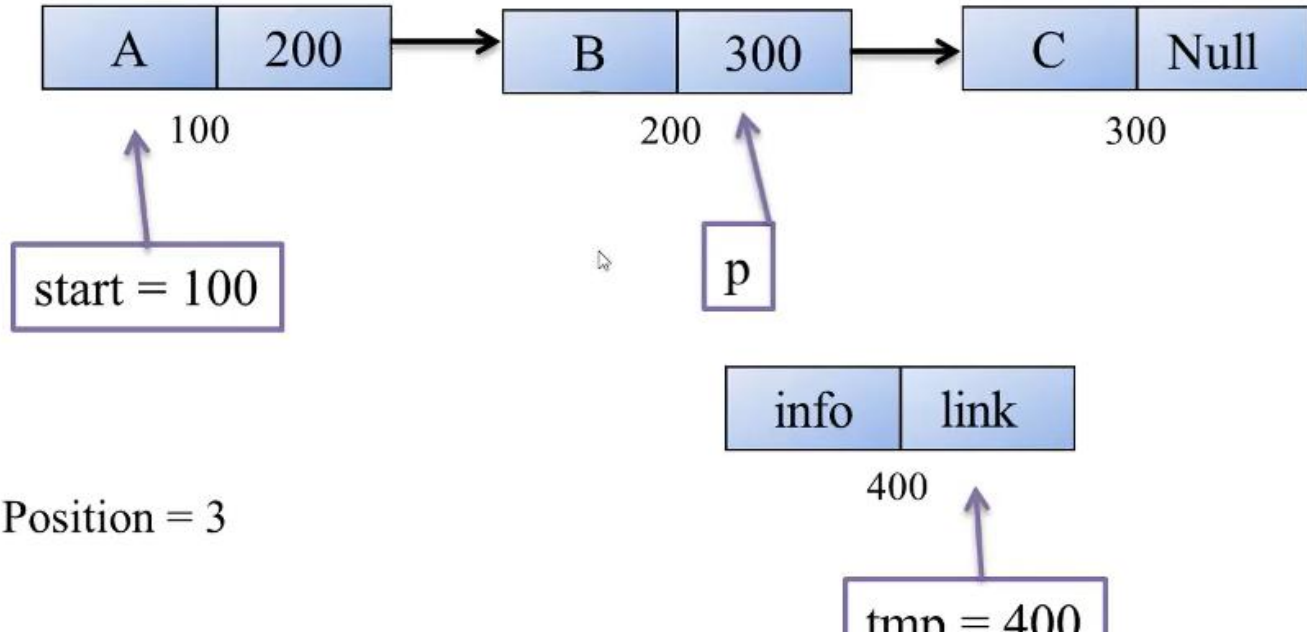


Position = 3



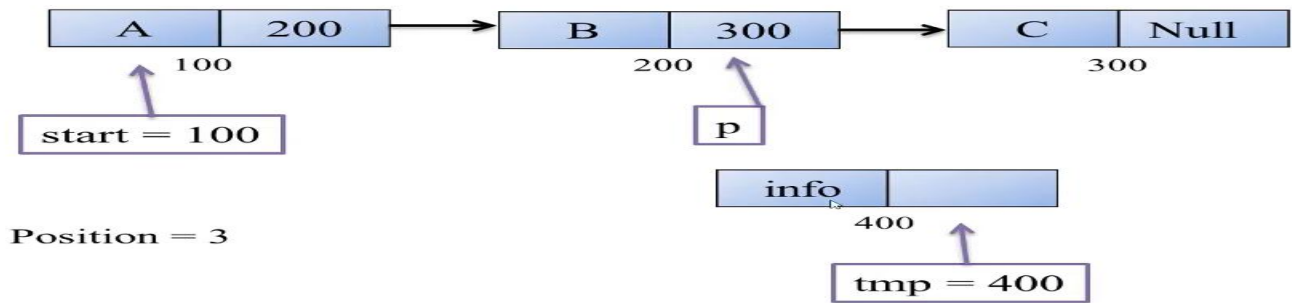
Singly Linked List

Insertion at nth Position

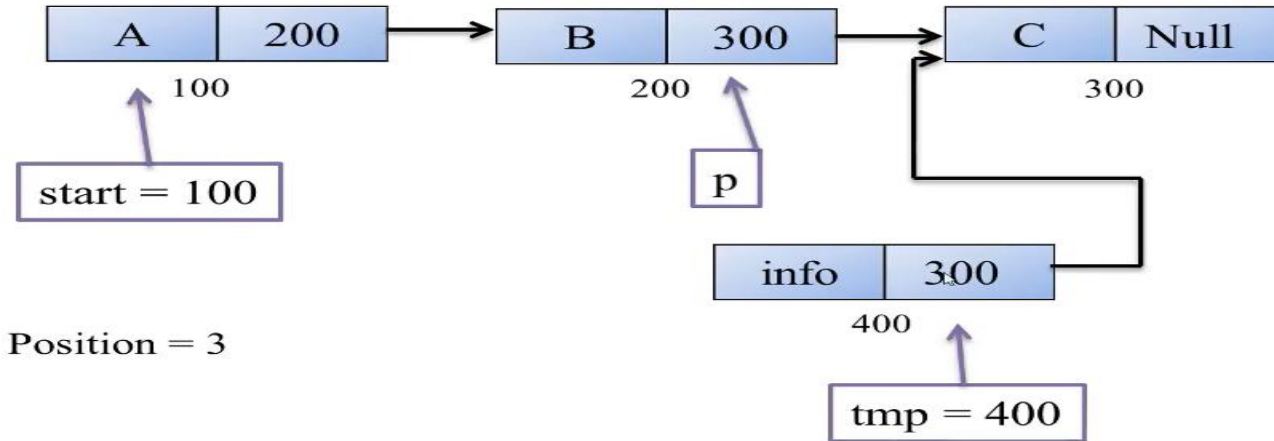


Replace link part with last node address

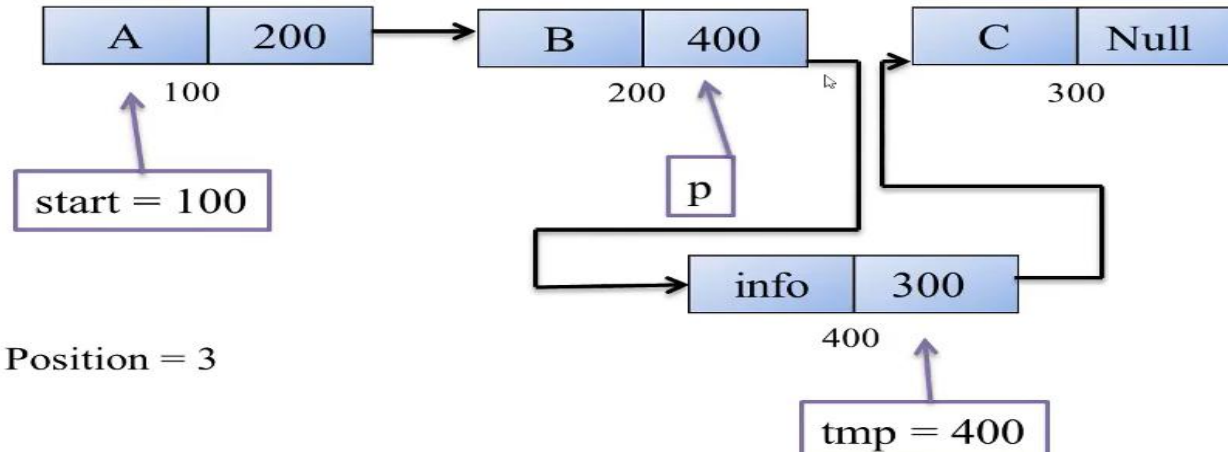
Singly Linked List Insertion at nth Position

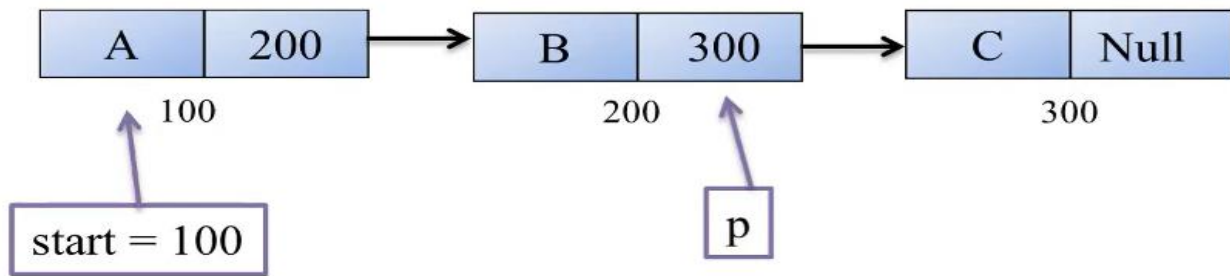


Singly Linked List Insertion at nth Position



Singly Linked List Insertion at nth Position

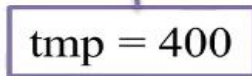




Position = 3



400

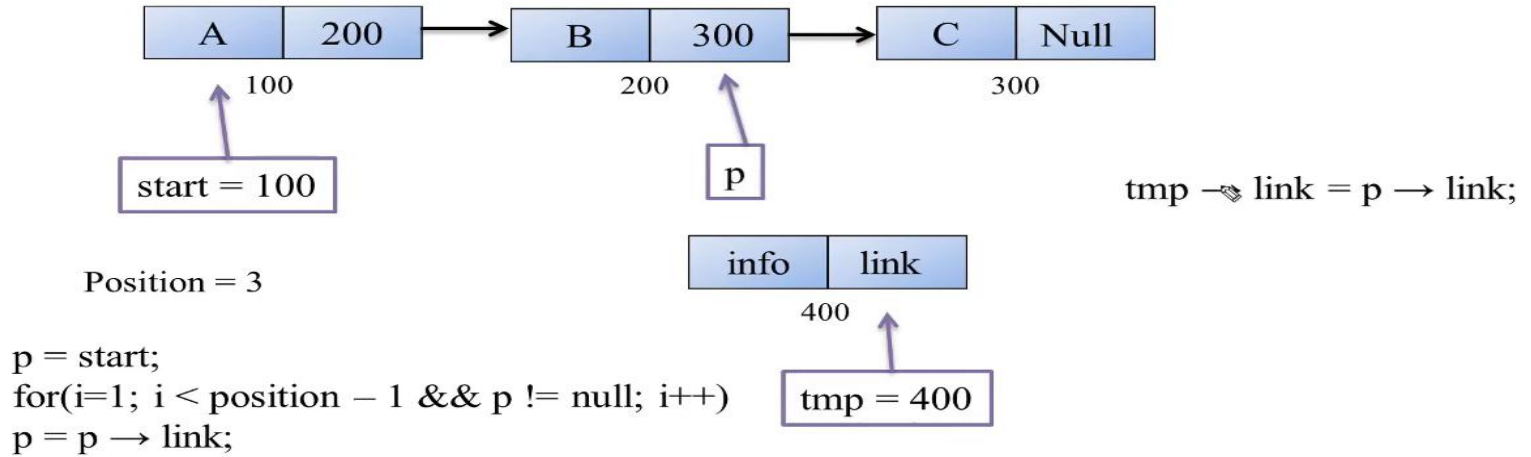


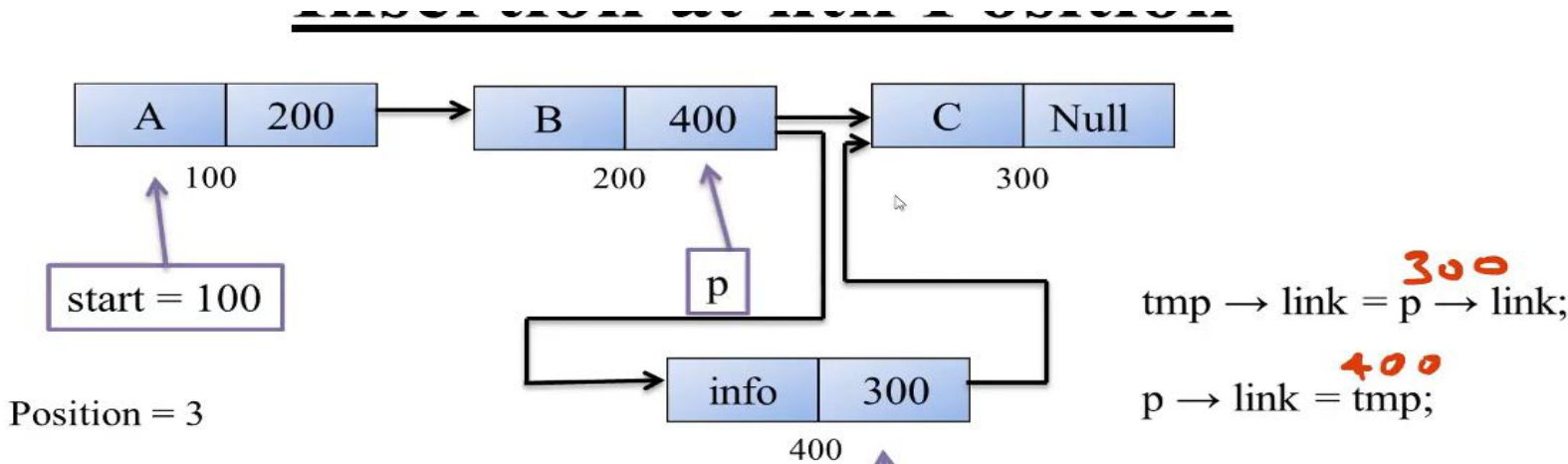
$p = \text{start};$ ✓ *1 < 2 2 2 1 = Null;*

for($i=1$; $i < \text{position} - 1 \ \&\& \ p \neq \text{null}; i++$)

✓ $p = p \rightarrow \text{link};$
p = 200

Insertion at nth Position





```
p = start;  
for(i=1; i < position - 1 && p != null; i++)  
p = p → link;
```

tmp = 400

tmp → link = p → link;
p → link = tmp;

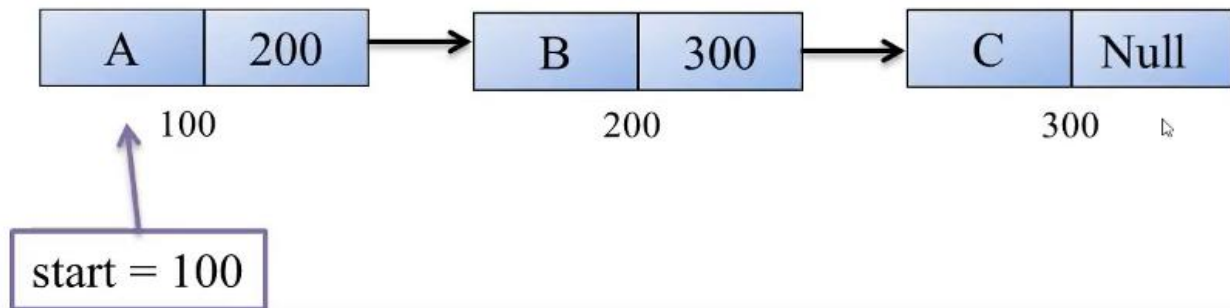
Single linked List At END

To insert a node at the end of the list, traverse the list until the last node is reached, and then link the new node to the current last node.

Step-by-step approach:

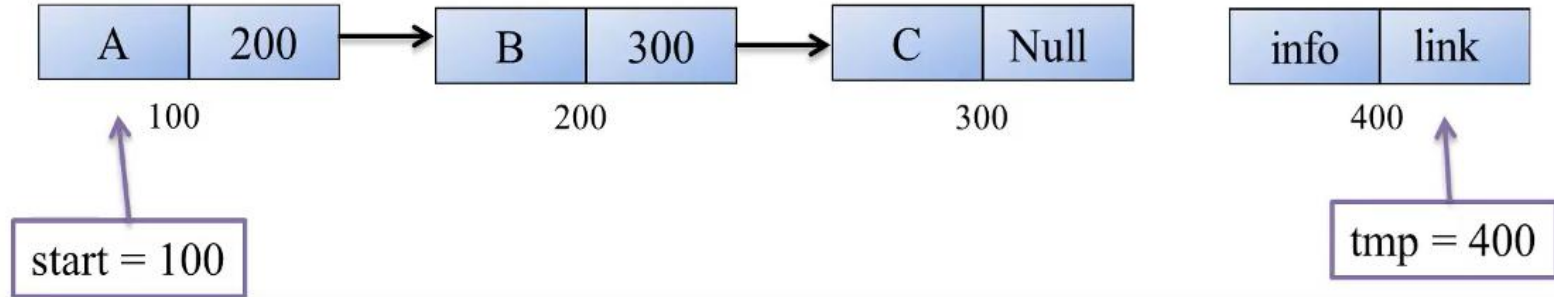
- Create a new node with the given value.
- Check if the list is empty:
 - If it is, make the new node the head and return.
- Traverse the list until the last node is reached.
- Link the new node to the current last node by setting the last node's next pointer to the new node.

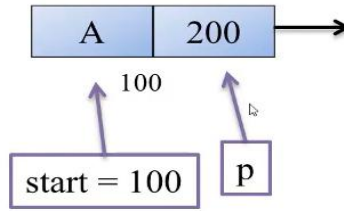
Singly Linked List Insertion at the End



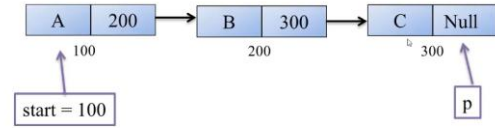
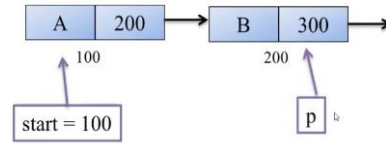
Singly Linked List

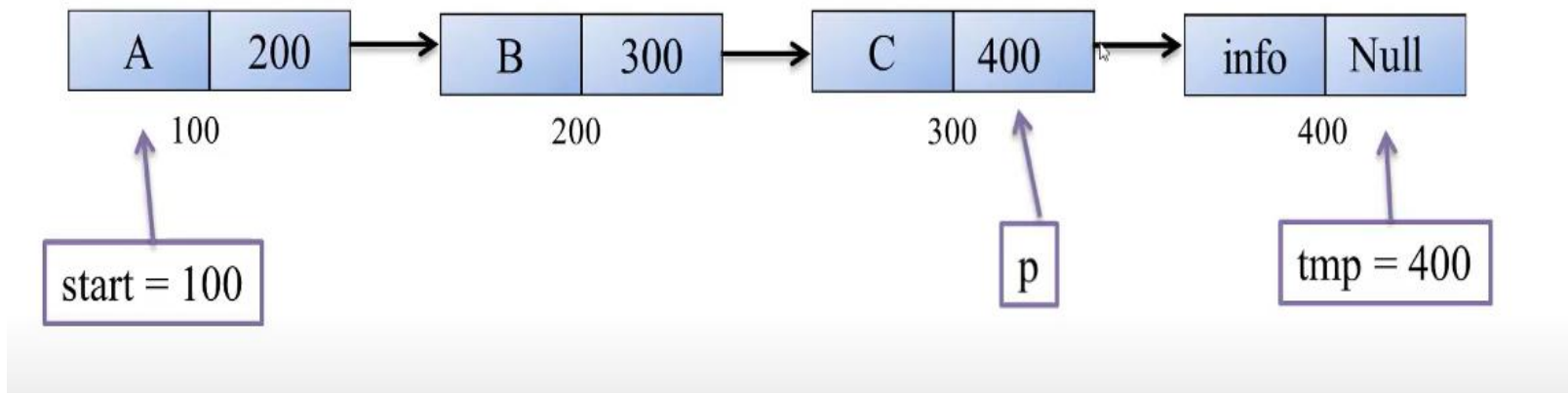
Insertion at the End

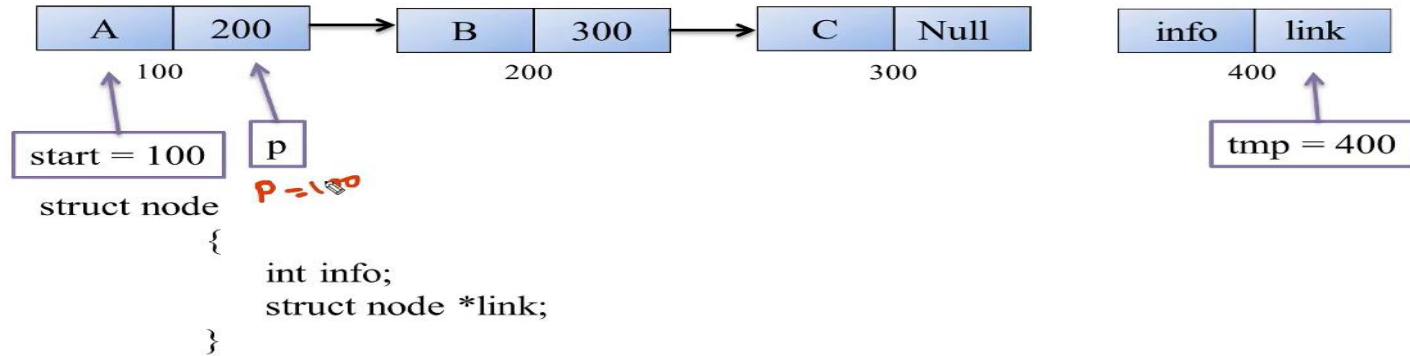




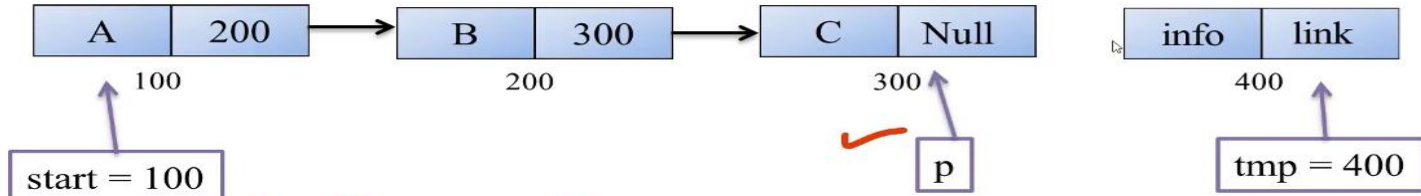
INSERTION







```
struct node *tmp;
tmp = (struct node *) malloc(sizeof(struct node));
struct node *p;
p = start;
```



```

struct node
{
    int info;
    struct node *link;
}

```

```

struct node *tmp;
tmp = (struct node *) malloc(sizeof(struct node));

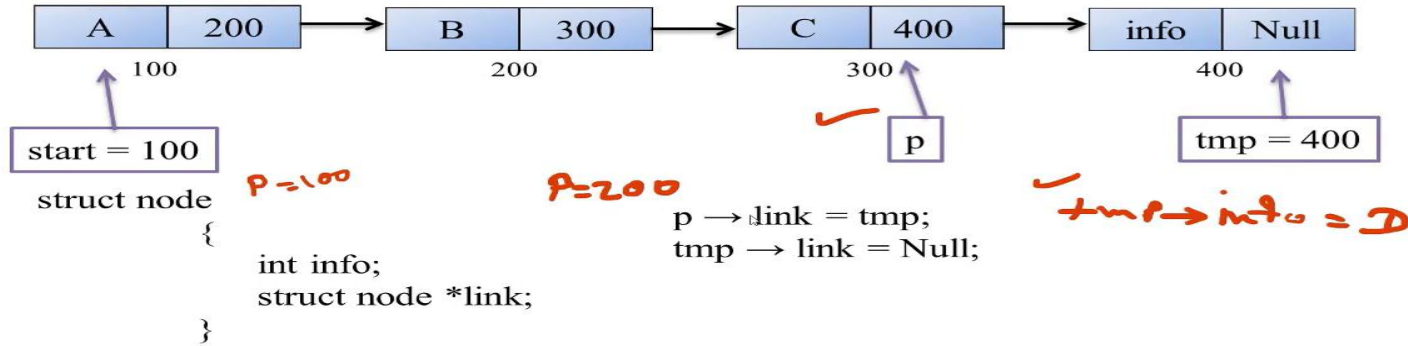
```

```

struct node *p;
p = start;
✓ While(p != Null)
    p = p → link;

```

P = 200
P = 300
P = NULL



```

struct node *tmp;
tmp = (struct node *) malloc(sizeof(struct node));

struct node *p;
p = start;
✓ While(p != Null)
  p = p -> link;
  
```

p = 200
p = 300
p = NULL

Link For Program(Single Linked List)

<https://www.programiz.com/online-compiler/9DMB5oaWI0qiN>

Deletion in Linked List

Deleting a node in a Linked List is an important operation and can be done in three main ways: removing the first node, removing a node in the middle, or removing the last node.

Types of Deletion in Linked List

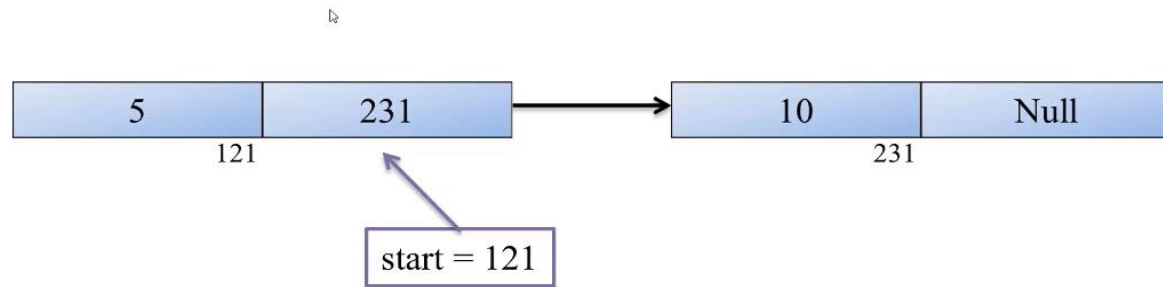
- Deletion at the Beginning of Linked List
- Deletion at Specific Position of Linked List
- Deletion at the End of Linked List

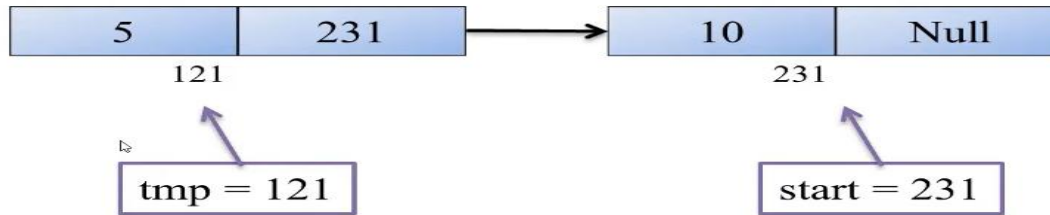
1. Deletion at the Beginning of Linked List

Deletion at the Beginning operation involves removing the first node of the linked list.

Step-by-Step Approach:

1. **Check if the list is empty:** If the head is NULL, the list is empty, and there's nothing to delete.
2. **Update the head pointer:** Set the head to the second node (`head = head->next`).
3. **Delete the original head node:** The original head node is now unreferenced, and it can be freed/deleted if necessary

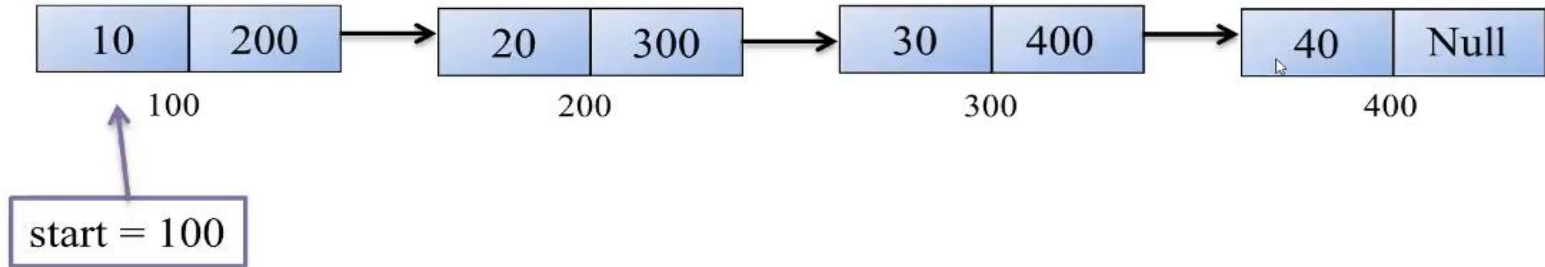




```
struct node *tmp;  
tmp = start;  
start = start → link;  
free (tmp);
```


Deletion At End(Last Node)

Singly Linked List Deletion at the End (Last node)



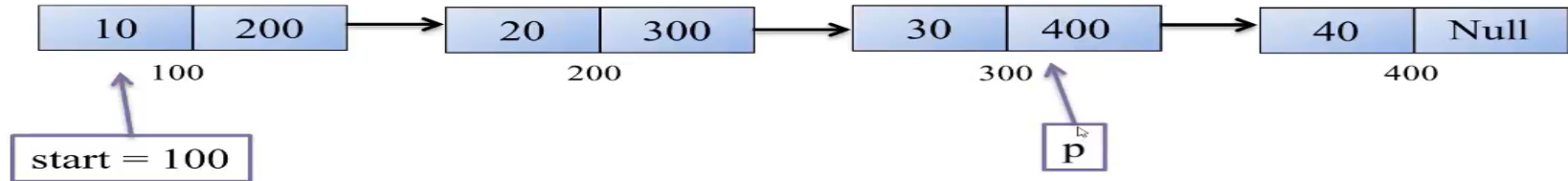
Deletion at the end operation involves removing the last node of the linked list.

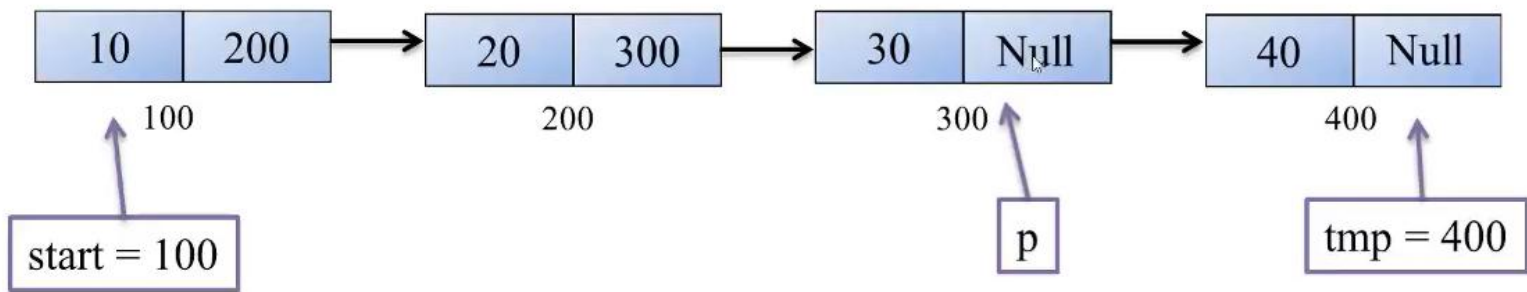
Step-by-Step Approach:

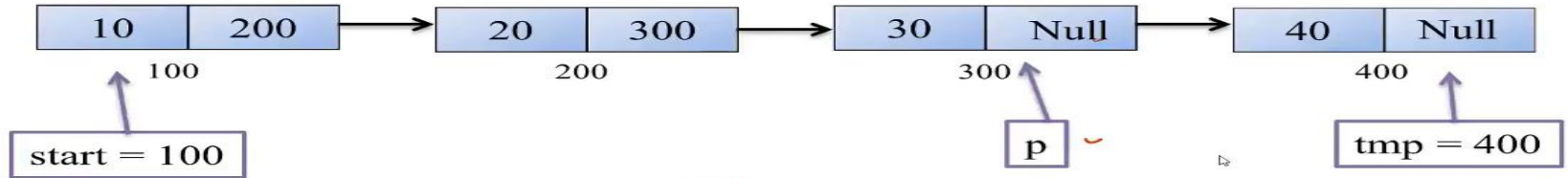
1. **Check if the list is empty:** If the head is NULL, the list is empty, and there's nothing to delete.
2. **If the list has only one node:** Simply set the head to NULL (the list becomes empty).
3. **Traverse the list** to find the second-last node: Start from the head and iterate through the list until you reach the second-last node (where the next of the node is the last node).
4. **Update the next pointer** of the second-last node: Set the second-last node's next to NULL (removing the link to the last node).
5. **Delete the last node:** The last node is now unreferenced and can be deleted or freed, depending on the language used.

Traverse a pointer upto last node

Singly Linked List Deletion at the End (Last node)







```

struct node *p;
struct node *tmp;
p = start;
while(p → link != Null)

```

```

{
  ✓ If(p → link → info == data)
  {
    ✓ 30 → info == 40 ✓
    tmp = p → link;
    p → link = tmp → link;
    free(tmp);
  }
  ✓ p = p → link
}

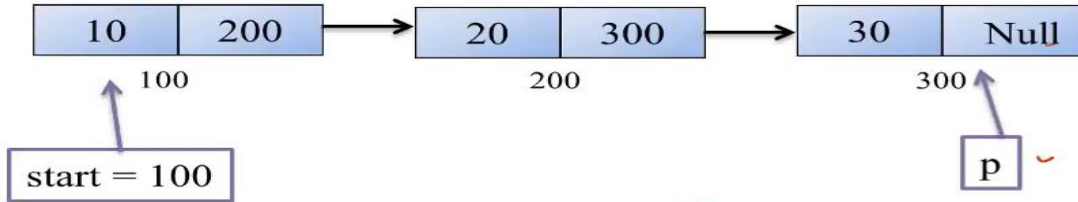
```

data = 40

① $p \rightarrow \text{link} = 200 \neq \text{Null}$
 $p = 200$

② $p \rightarrow \text{link} = 300 \neq \text{Null}$
 $p = 300$

③ $p \rightarrow \text{link} = 400 \neq \text{Null}$
 $tmp = 400$
 $p \rightarrow \text{link} = \text{Null}$
 $\text{free}(tmp)$



```

struct node *p;
struct node *tmp;
p = start;

```

```

while(p → link != Null)

```

```

{
    ✓ If(p → link → info == data)
    {
        ✓ tmp = p → link;
        p → link = tmp → link;
        free(tmp);
    }
    ✓ p = p → link
}

```

data = 40

④ ~~Null~~ != null

① $P \rightarrow \text{link} = 200 \neq \text{Null}$
 $P = 200$

② $P \rightarrow \text{link} = 300 \neq \text{Null}$
 $P = 300$

③ $P \rightarrow \text{link} = 400 \neq \text{Null}$
 $\text{tmp} = 400$
 $P \rightarrow \text{link} = \text{Null}$
 $\text{free}(\text{tmp})$
 $P = \text{Null}$

Deletion at Specific Position of Linked List

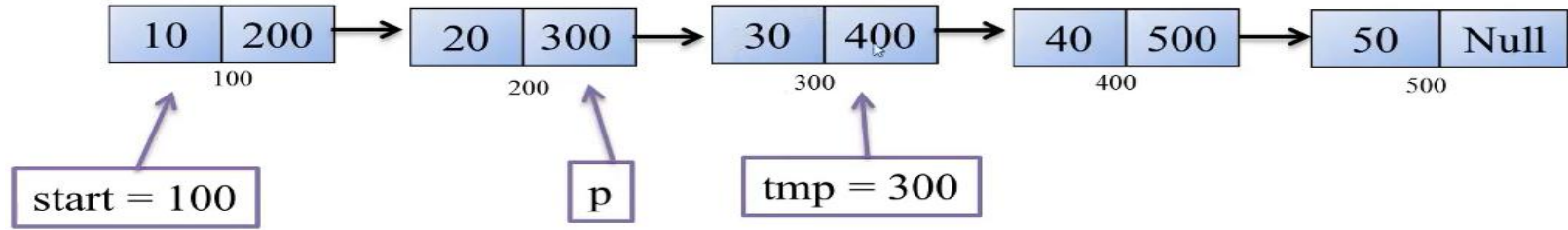
Deletion at a specified position in a linked list involves removing a node from a specific index/position, which can be the first, middle, or last node.

Step-by-Step Approach:

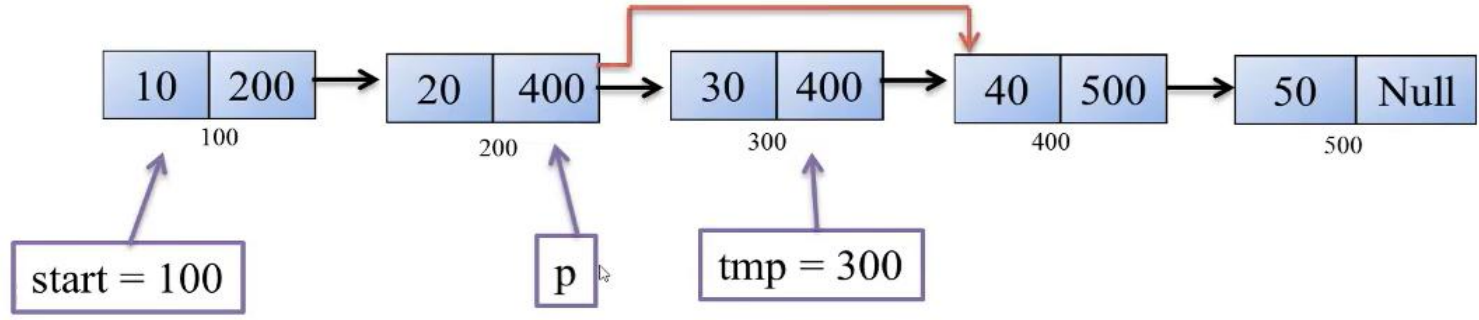
1. **Check if the position is valid:** If the position is out of bounds (greater than the length of the list), return an error or handle appropriately.
2. **Traverse the list** to find the node just before the one to be deleted: Start from the head and move through the list until reaching the node at position $n-1$ (one before the target position).
3. **Update the next pointer:** Set the next pointer of the $(n-1)^{th}$ node to point to the node after the target node (`node_to_delete->next`).
4. **Delete the target node:** The node to be deleted is now unreferenced, and in languages like C++ or Java, it can be safely deallocated.

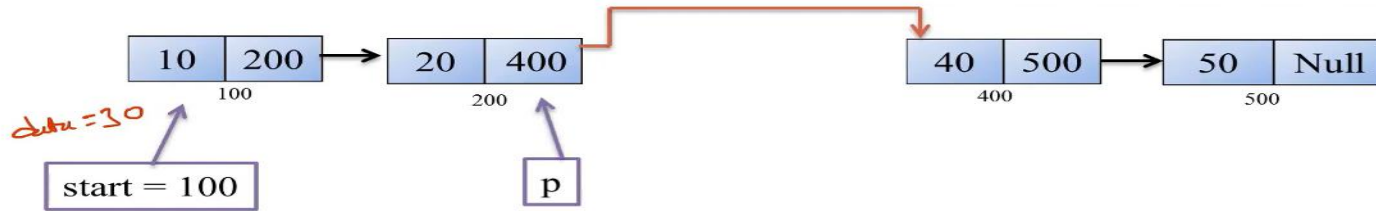
data = 30

DELETION OF A NODE



data = 30





```

struct node *p;
struct node *tmp;
p = start;
while(p → link != Null)
{
    ✓ If(p → link → info == data)
    {
        ✓ 300 → info 20 == 30
        tmp = p → link;
        p → link = tmp → link;
        free(tmp);
    }
    ✓ p = p → link
}

```

① $P \rightarrow \text{link} = 200 \neq \text{Null} \checkmark$
 $P = P \rightarrow \text{link} =$
 $P = 200$

② $P \rightarrow \text{link} = 300 \neq \text{Null}$
 $\text{tmp} = 300$
 $P \rightarrow \text{link} = 400$

Link for a program (Deleting Link List)

<https://www.programiz.com/online-compiler/1gfl9AS3ouYC1>

Reverse a Linked List

It has a two methods

1.Iteration Method

2.Recursive Method

Reverse a Linked List (Iteration Method)

*The idea is to reverse the links of all nodes using **three** pointers:*

- ***prev**: pointer to keep track of the previous node*
- ***curr**: pointer to keep track of the current node*
- ***next**: pointer to keep track of the next node*

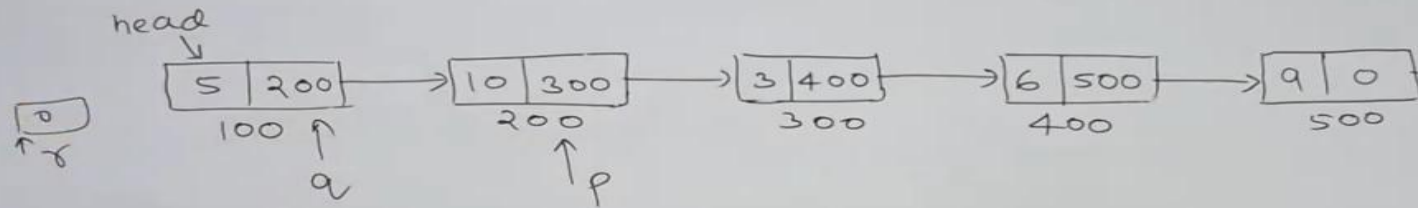
*Starting from the first node, initialize **curr** with the head of linked list and **next** with the next node of curr. Update the next pointer of **curr** with **prev**. Finally, move the three pointer by updating **prev** with **curr** and **curr** with **next**.*

Follow the steps below to solve the problem:

- Initialize three pointers **prev** as NULL, **curr** as **head**, and **next** as NULL.
- Iterate through the linked list. In a loop, do the following:
 - Store the next node, **next = curr -> next**
 - Update the next pointer of curr to prev, **curr -> next = prev**
 - Update prev as curr and curr as next, **prev = curr** and **curr = next**

Follow the steps below to solve the problem:

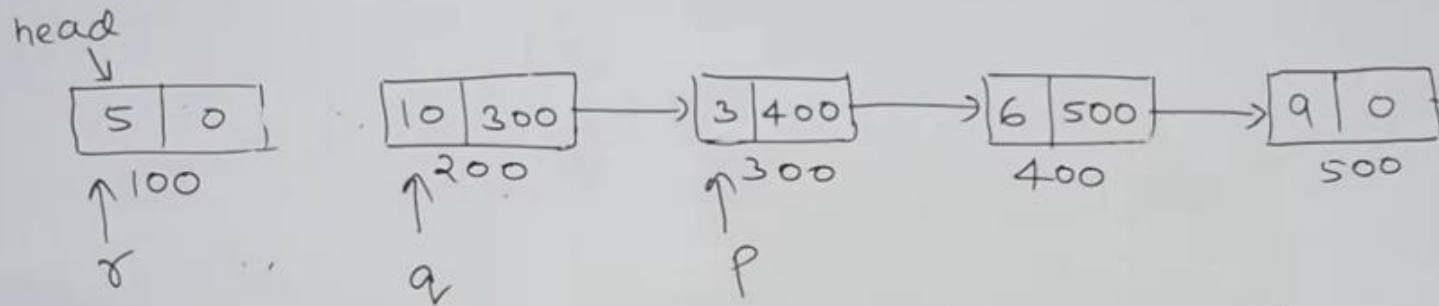
- Initialize three pointers **prev** as NULL, **curr** as **head**, and **next** as NULL.
- Iterate through the linked list. In a loop, do the following:
 - Store the next node, **next = curr -> next**
 - Update the next pointer of curr to prev, **curr -> next = prev**
 - Update prev as curr and curr as next, **prev = curr** and **curr = next**



```
struct node  
{ int data;  
  struct node * next;  
};
```

Reverse Linked List.

```
struct node * p, * q, * r;  
p = head->next;  
q = head;  
r = NULL;
```



```
while (p != NULL)
{
    q->next = r;
```

```
    r = q;
```

```
    q = p;
```

```
    p = p->next;
```

```
struct node
```

```
{ int data;
```

```
  struct node * next;
```

```
};
```

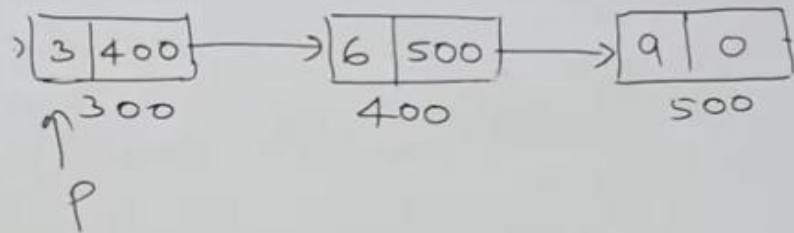
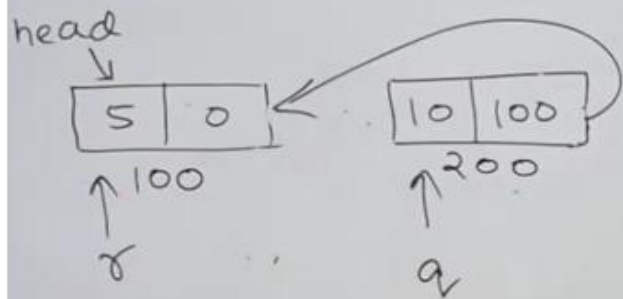
```
Reverse Linked List.
```

```
struct node * p, * q, * r;
```

```
p = head->next;
```

```
q = head;
```

```
r = NULL;
```



```
while (p != NULL)
{
    q->next = r;
```

```
    r = q;
```

```
    q = p;
```

```
    p = p->next;
```

struct node

```
{ int data;
```

```
  struct node * next;
```

};

Reverse Linked List

```
struct node * p, * q, * r;
```

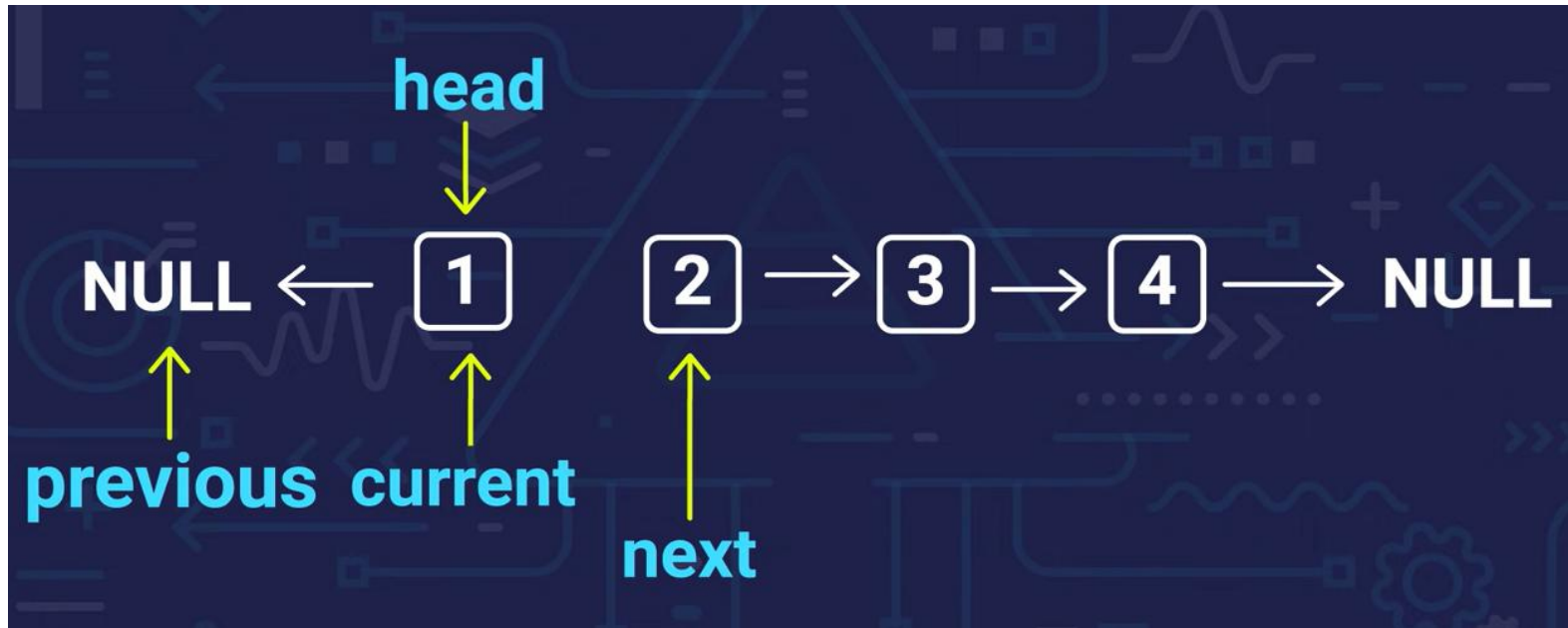
```
p = head->next;
```

```
q = head;
```

```
r = NULL;
```











Reverse a Linked List (Recursive Method)

*The idea is to reach the **last node** of the linked list using [recursion](#) then start **reversing** the linked list from the last node.*

Follow the steps below to solve the problem:

- Divide the list in two parts – first node and rest of the linked list.
- Call **reverse** for the rest of the linked list.
- Link the rest linked list to first.
- Fix head pointer to NULL.

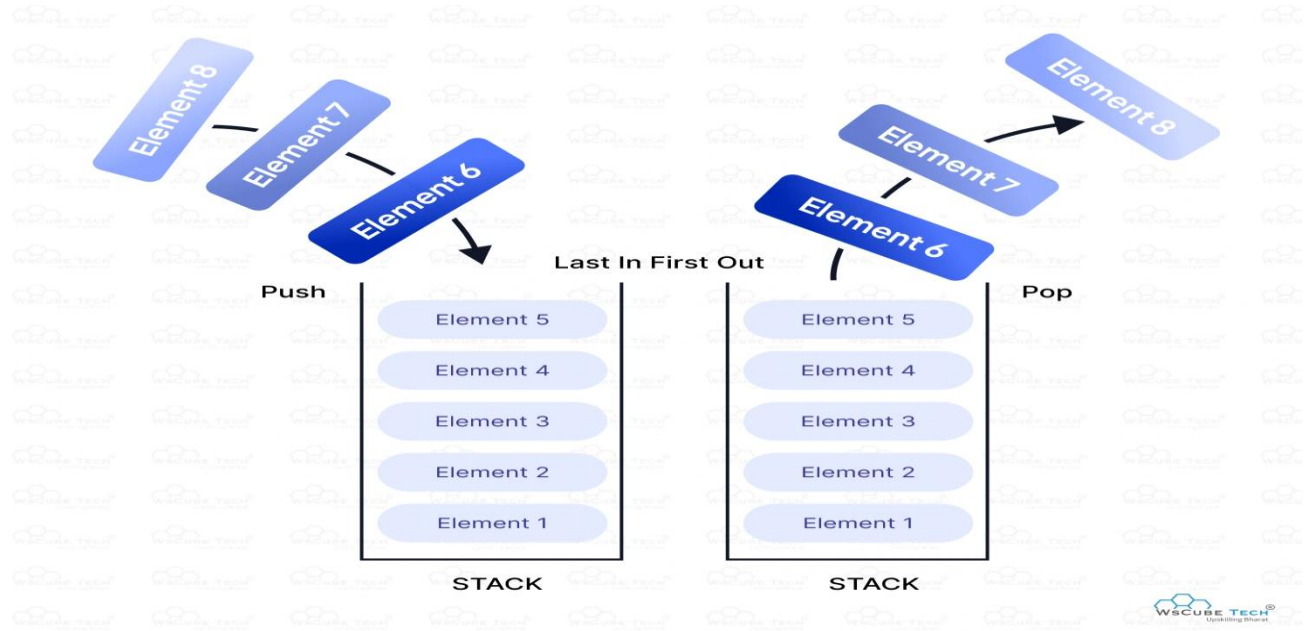






Stack using Linked List

Stack is a linear data structure that follows **LIFO (Last In First Out) Principle**, the last element inserted is the first to be popped out. It means both insertion and deletion operations happen at one end only.



Stack Operations With Example

We need to follow certain operations that are essential for managing data within a stack:

1. Push

The push operation adds an element to the top of the stack. This is the primary way to insert data into the stack.

E

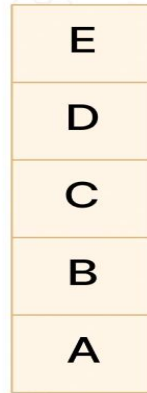
Push Operation

Top →



Stack

Top →

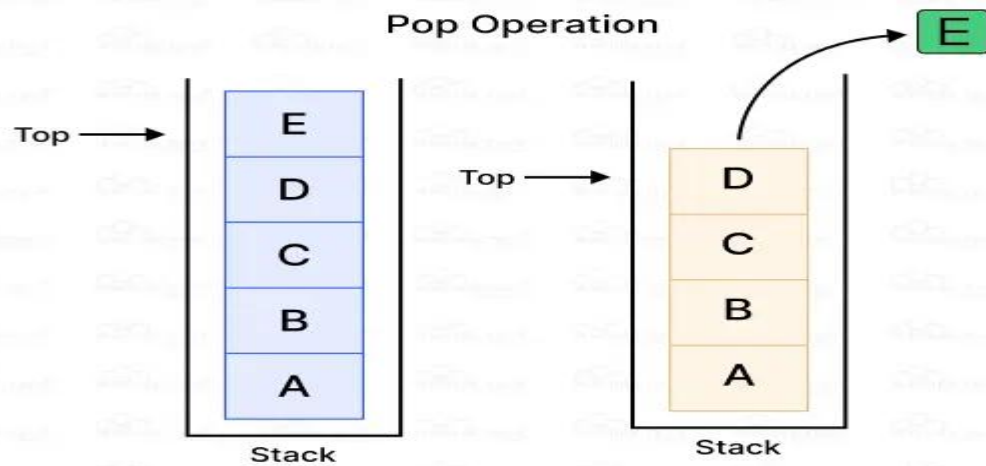


Stack

2. Pop

The pop operation removes the top element from the stack and returns it. This operation follows the LIFO principle, ensuring that the last element added is the first to be removed.

Pop Operation



Stack Using Linked List in C

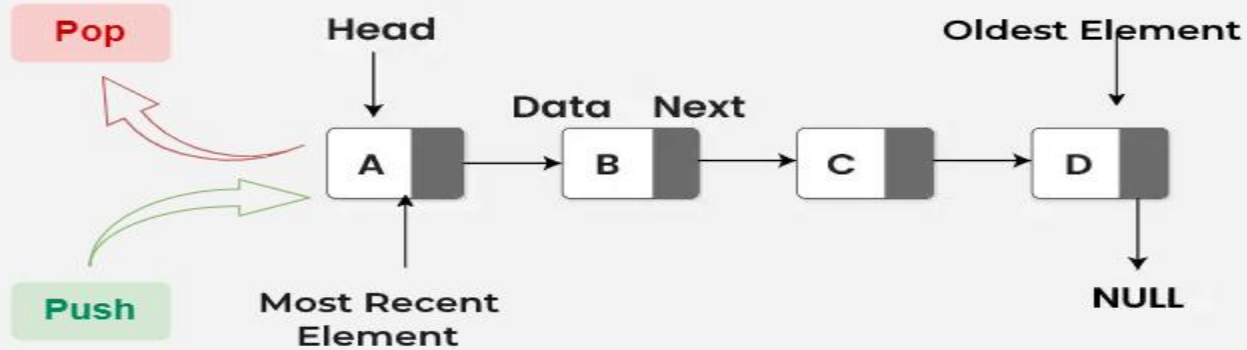
Stack is a linear data structure that follows the Last-In-First-Out (LIFO) order of operations. This means the last element added to the stack will be the first one to be removed. There are different ways using which we can implement stack data structure in C.

Implementation of Stack using Linked List in C

Stack is generally implemented using an array but the limitation of this kind of stack is that the memory occupied by the array is fixed no matter what are the number of elements in the stack. In the stack implemented using linked list implementation, the size occupied by the linked list will be equal to the number of elements in the stack. Moreover, its size is dynamic. It means that the size is gonna change automatically according to the elements present.

In C, the stack that is implemented using a linked list can be represented by the pointer to the head node of the linked list. Each node in that linked list represents the element of the stack. The type of linked list here is a singly linked list in which each node consists of a data field and the next pointer.

Stack as Linked List



Basic Operations of Linked List Stack in C

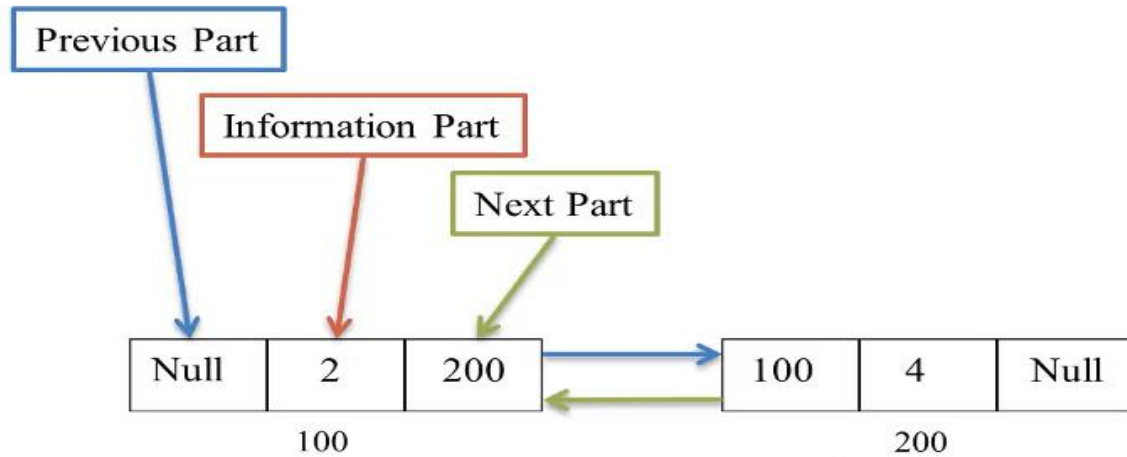
Operation	Description	Time Complexity	Space Complexity
isEmpty	Returns true if the stack is empty, false otherwise.	O(1)	O(1)
Push	This operation is used to add/insert/push data into the stack.	O(1)	O(1)
Pop	This operation is used to delete/remove/pop data into the stack.	O(1)	O(1)
Peek	This operation returns the top element in the stack.	O(1)	O(1)

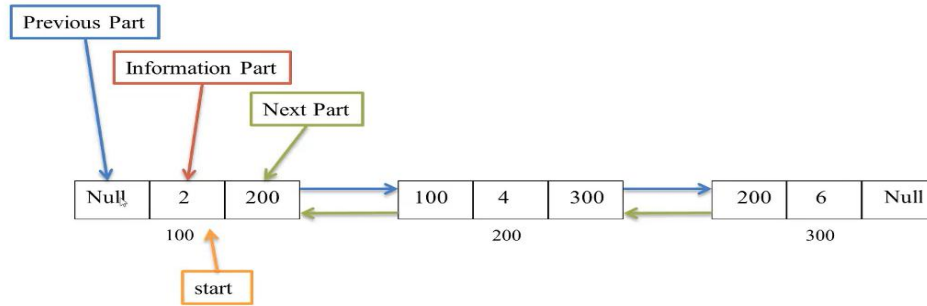
Double Linked List

It is the linear collection of data item called node where each node has divided into three part i.e. data part, previous part and next part.

Data part store the data item.
Previous part store address of previous node and next part store address of next node.

It starts with special pointer called first pointer and ending with last pointer.





Pointer **Start** Points First/Head Node

Previous Part of First Node and Next Part of Last Node will be always Null

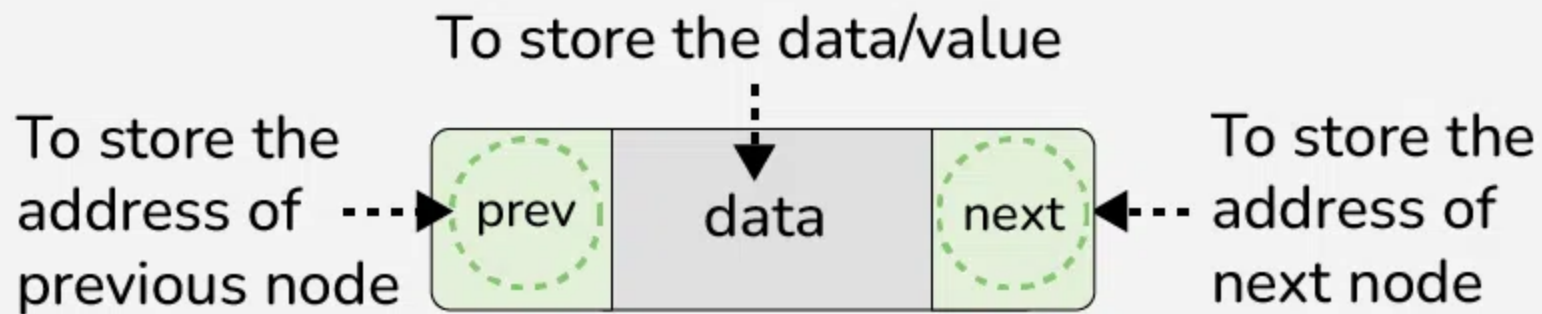
Next Part of Each Node contain Address of Successor Node

Previous Part of Each Node contain Address of Predecessor Node

If a node contains Null in the Previous Part it means it is First Node of the list



Doubly Linked List



Node Structure of Doubly Linked List

Traversal in Doubly Linked List

Traversal in a **Doubly Linked List** involves visiting each node, processing its data, and moving to the next or previous node using the forward (next) and backward (prev) pointers.

Step-by-Step Approach for Traversal:

1. **Start from the head** of the list.
2. **Traverse forward:**
 - Visit the current node and process its data (e.g., print it).
 - Move to the next node using `current = current->next`.
 - Repeat the process until the end of the list (`current == NULL`).
3. **Optionally, traverse backward:**
 - Start from the tail (last node).
 - Visit the current node and process its data.
 - Move to the previous node using `current = current->prev`.
 - Repeat the process until the beginning of the list (`current == NULL`).

For e.g:- Text:

Data - HNISH (Forward & Backward)

Tab

add	Previous	Data	Next
1	NULL	H	2
2	1	N	3
3	2	I	4
4	3	S	5
5	4	H	NULL
6			7
7			8
8			NULL

Algorithm for Forward Traversing:

1. Begin
2. Set $i \leftarrow \text{first} \rightarrow i = 1$ (Start from the first node)
3. Repeat steps 4 & 5 until $i = \text{NULL}$
4. Print $\text{data}[i]$
5. Move to the next node ($i \leftarrow \text{next}[i]$)
6. Exit when $i = \text{NULL}$

This algorithm describes the process of **traversing a doubly linked list** in the forward direction, starting from the first node and moving sequentially to the next node until reaching NULL.

Step-by-step Execution:

Step	i (Current Node)	data[i] (Printed)	next[i] (Move to)
1	1	H	2
2	2	N	3
3	3	I	4
4	4	S	5
5	5	H	NULL
6	NULL (End)	-	-

Algorithm for Backward Traversing

Step 1: Begin

Step 2: Set $i \leftarrow \text{last}$ (5)

Step 3: Repeat steps 4 & 5 until $i = \text{NULL}$

Step 4: Print $\text{data}[i]$

Step 5: $i \leftarrow \text{previous}[i]$

Step 6: Exit

This algorithm describes the process of **traversing a doubly linked list in reverse order**, starting from the **last node** and moving to the **previous node** until reaching NULL.

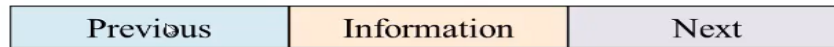
Doubly Linked List

In this list each and every node is connected with one another in both direction so, the traversal can be done from head node to tail node and vice-versa.

Here each node has been divided into three parts : -

(i) Information Part

(ii) Left Pointer

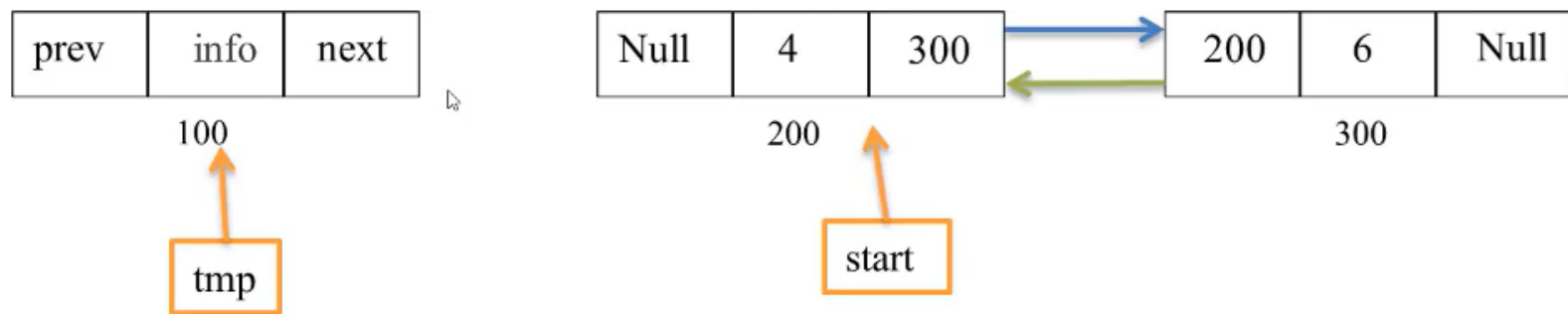


(iii) Right Pointer

- Information Part/ Data Part – It holds any type of value which may be numeric or character.
- Left Pointer/Previous Part – It holds the address of predecessor node/ Previous node.
- Right Pointer/ Next Part – It holds the address of successor node/ Next node.

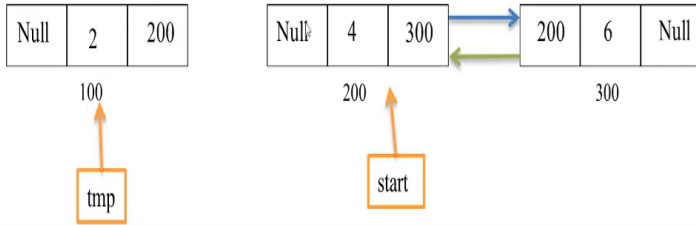
Doubly Linked List

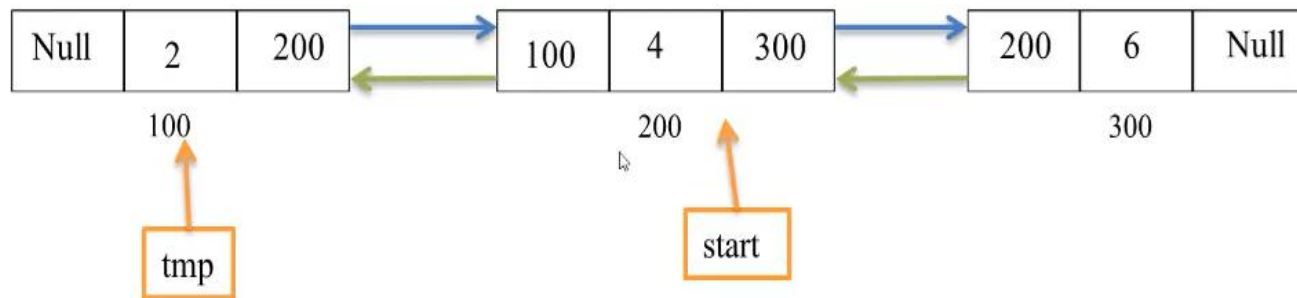
Insertion at the beginning



Doubly Linked List

Insertion at the beginning



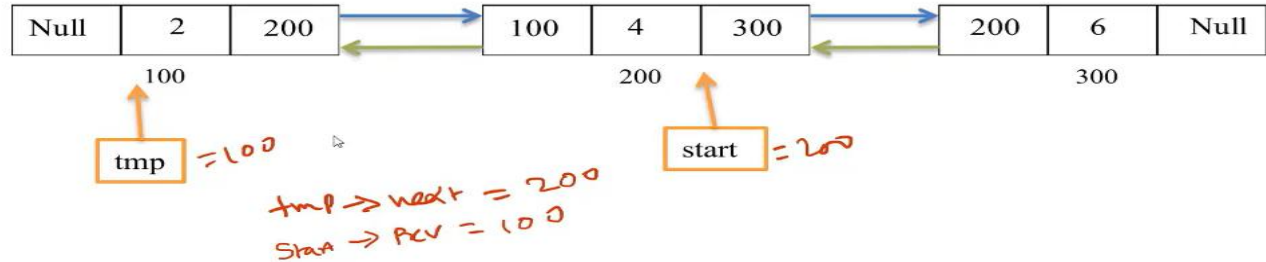


Insertion at the beginning

struct node

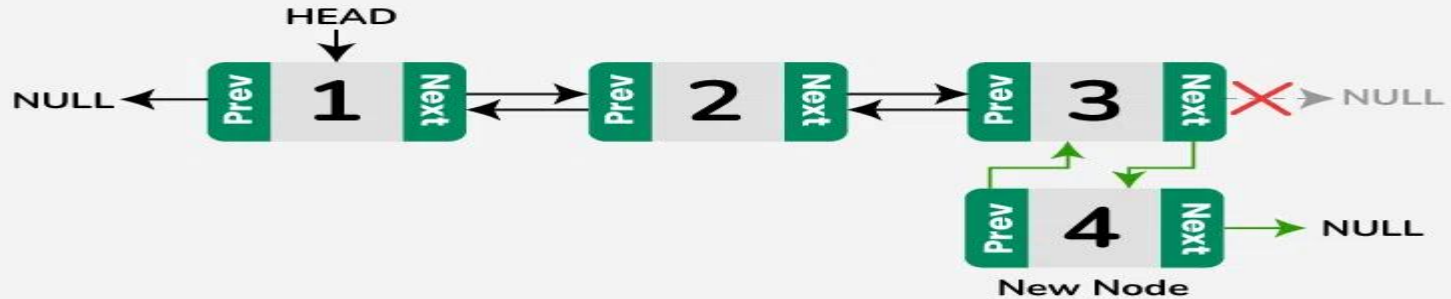
```
{  
    int info;  
    struct node *prev;  
    struct node *next;  
}
```

```
struct node *tmp;  
tmp = (struct node *) malloc(sizeof(struct node))  
tmp → info = 2  
tmp → prev = NULL  
tmp → next = start  
start → prev = tmp
```



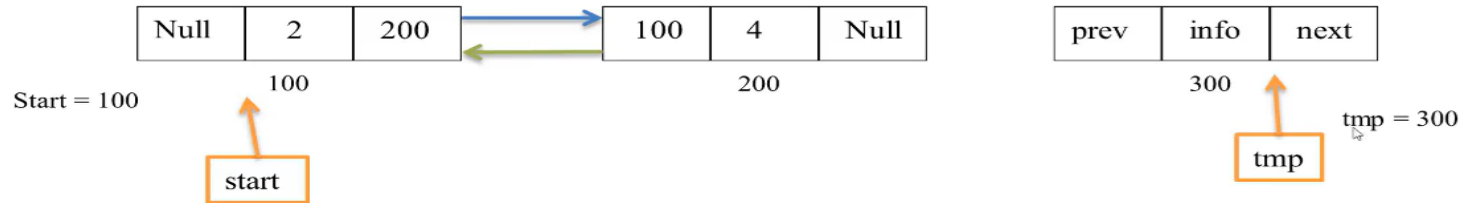
Insertion at the End in Doubly Linked List

Insertion at the end of a doubly linked list involves adding a new node after the last node. The new node's prev pointer points to the current last node, and its next pointer is set to NULL (since it will be the new last node). The next pointer of the current last node is updated to point to the new node. Finally, the tail pointer of the list is updated to point to the new node.

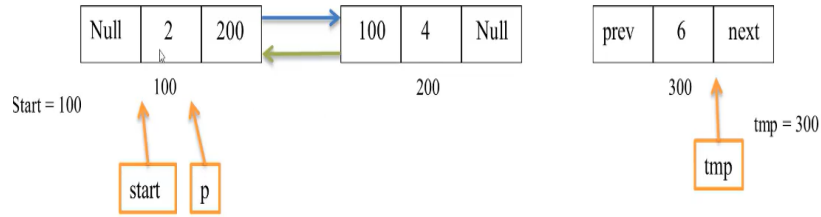


Insertion at the End in Doubly Linked List

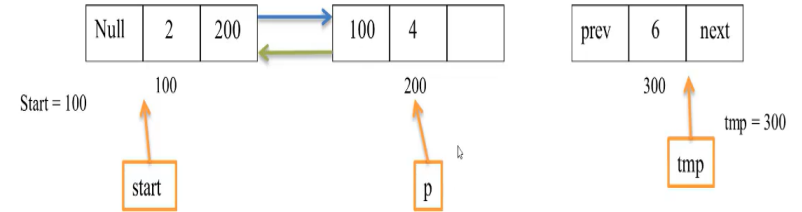
Doubly Linked List Insertion at the End



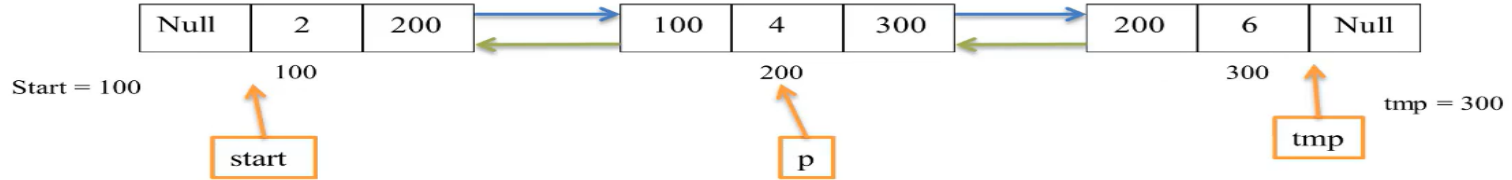
Doubly Linked List Insertion at the End



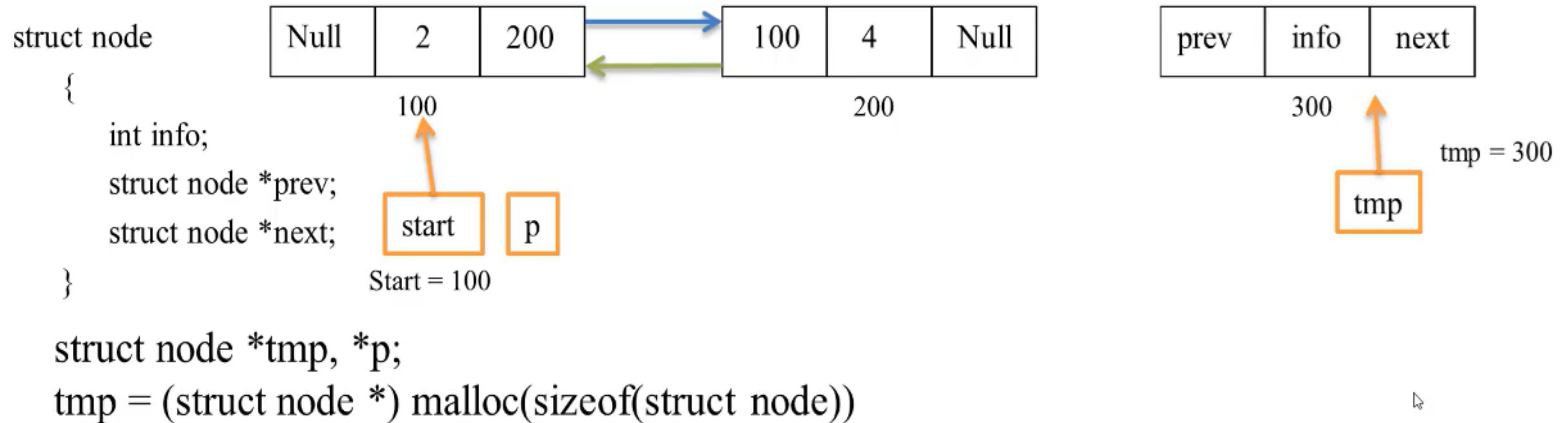
Doubly Linked List Insertion at the End



Doubly Linked List Insertion at the End



Insertion at the End



Insertion at the End

struct node

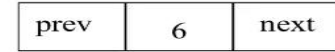
```
{
    int info;
    struct node *prev;
    struct node *next;
}
```



100
↑
start
Start = 100

P = 100

200
↑
p
P = 200



300
↑
tmp

tmp = 300

```
struct node *tmp, *p;
tmp = (struct node *) malloc(sizeof(struct node))
tmp → info = 6
p = start ✓
while(p → next != Null) ✓
    p = p → next
```

*I. P = 100
while (200 != null) ✓
P = P → next
P = 200*

*II. P = 200
while (Null != null) ✗*

Insertion at the End

struct node

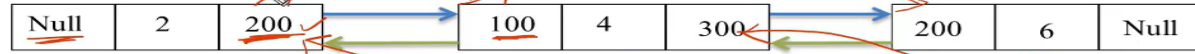
{

int info;

struct node *prev;

struct node *next;

}



100

start

Start = 100

p

tmp

tmp = 300

struct node *tmp, *p;

tmp = (struct node *) malloc(sizeof(struct node))

tmp → info = 6

p = start

while(p → next != Null)

p = p → next

p → next = tmp

tmp → next = Null

tmp → prev = p

p → next = 300

tmp → next = Null

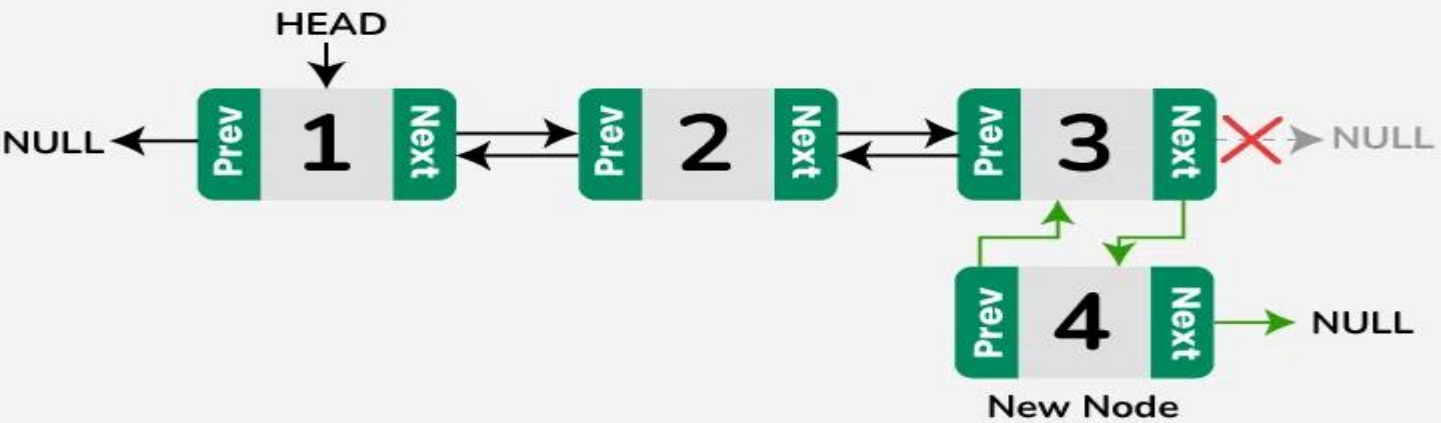
tmp → prev = 200

Insertion at the End in Doubly Linked List

To insert a new node at the end,

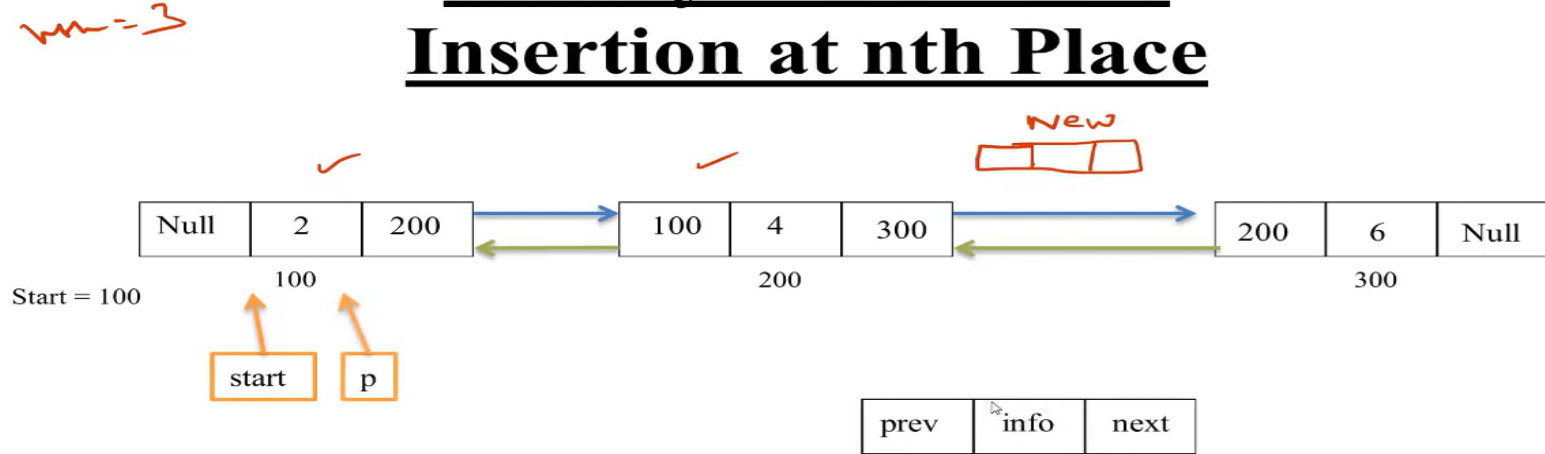
- Allocate memory for a new node, say **new_node** and assign the provided value to its data field.
- Initialize the next pointer of the new node to NULL, **new_node->next = NULL**.
- If the linked list is empty, we set the new node as the head of linked list and return it as the new head of the linked list.
- Traverse the entire list until we reach the last node, say **curr**.
- Set the next pointer of last node to new node, **curr->next = new_node**
- Set the prev pointer of new node to last node, **new_node->prev = curr**

Insertion at the End in Doubly Linked List



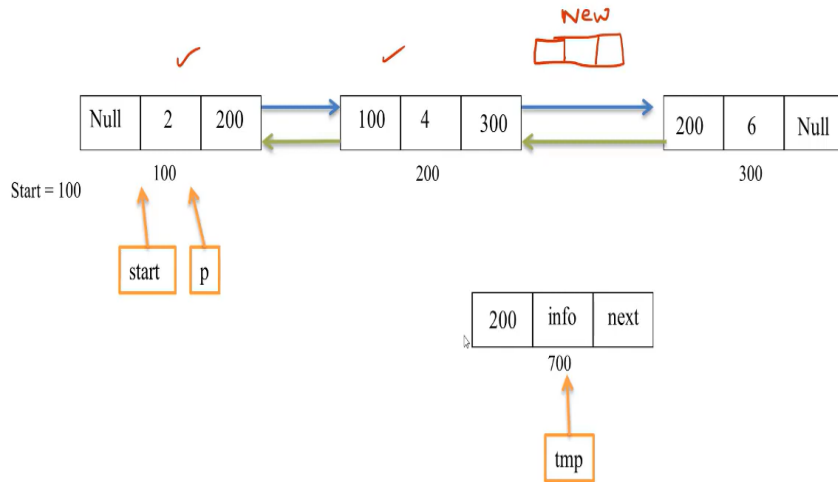
Insertion at the End in Doubly Linked List

Doubly Linked List Insertion at nth Place



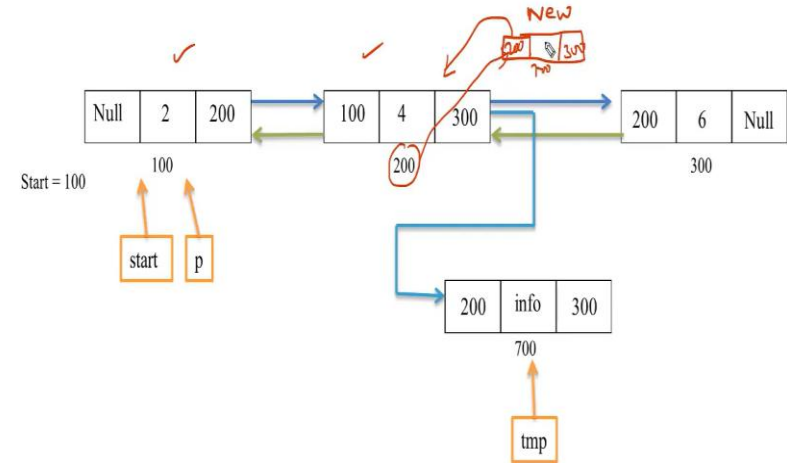
Doubly Linked List Insertion at nth Place

nm=3



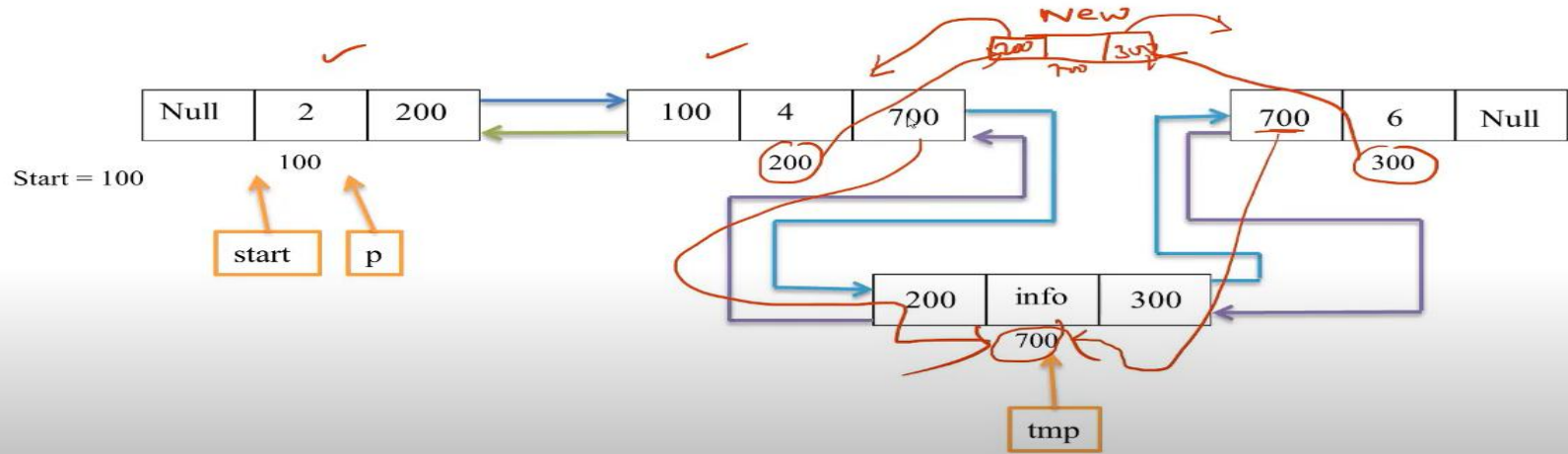
Doubly Linked List Insertion at nth Place

nm=3



mn=3

Doubly Linked List Insertion at nth Place

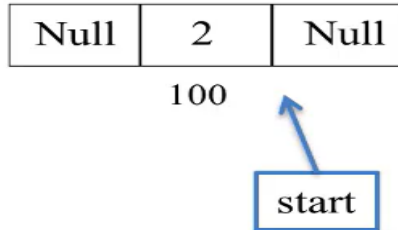


Deletion

- Deletion of the only node
- Deletion at the beginning (first node)
- Deletion at the End (last node)
- Deletion of the n^{th} node

Doubly Linked List

Deletion of the only node



Doubly Linked List

Deletion of the only node



100



start

start = Null

Deletion of the only node

```
struct node *tmp;  
tmp = start;  
start = NULL;  
free(tmp);
```



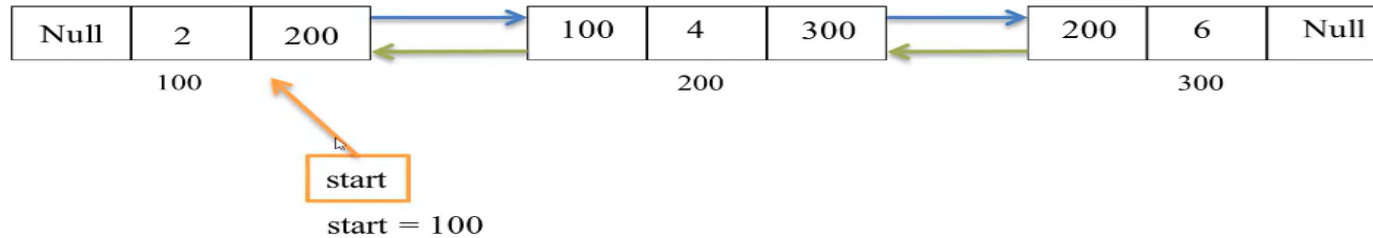
100



start = Null

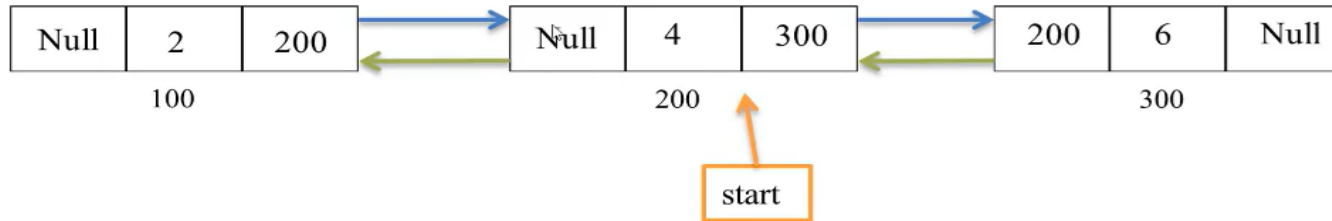
Doubly Linked List

Deletion at the beginning (First Node)

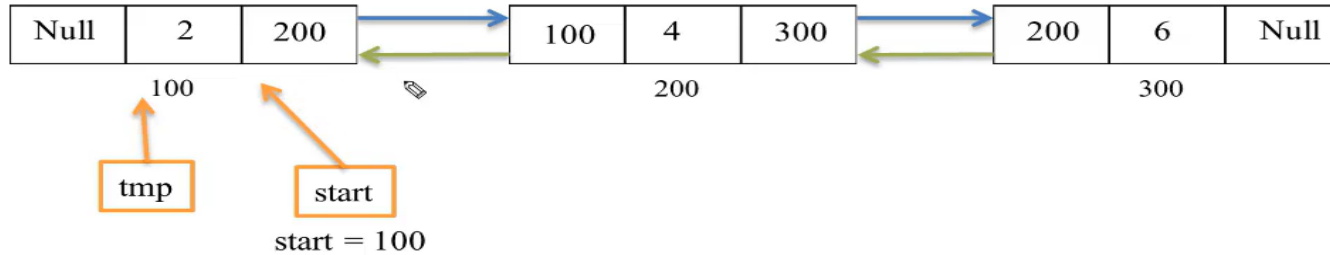


Doubly Linked List

Deletion at the beginning (First Node)

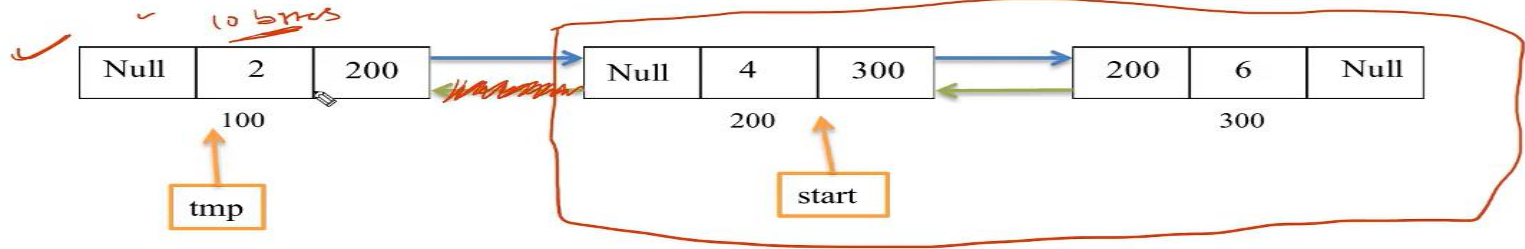


Deletion at the beginning (First Node)



```
struct node *tmp;  
tmp = start  
start = start → next
```

Deletion at the beginning (First Node)



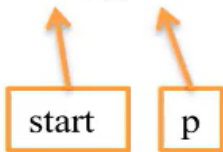
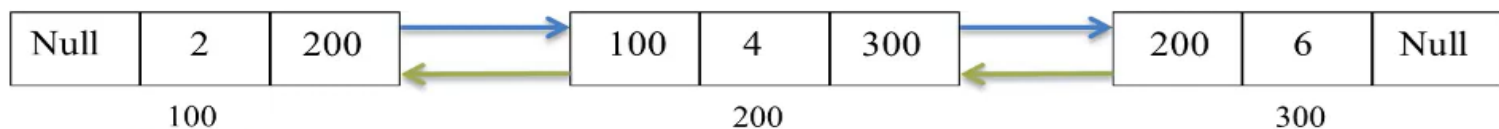
```
struct node *tmp;  
tmp = start  
start = start → next ✓  
start → prev = NULL  
free(tmp) ✓
```

Start = 100 ✓

Start = 200

Doubly Linked List

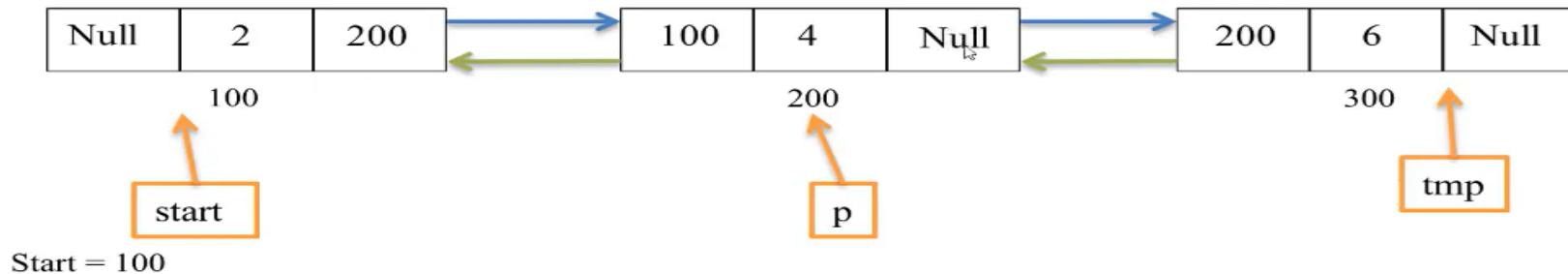
Deletion at the End (Last Node)



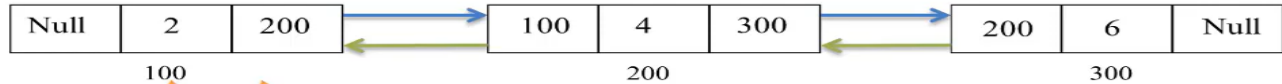
Start = 100

Doubly Linked List

Deletion at the End (Last Node)



Deletion of Last Node



start

p

tmp

```

struct node *p;
struct node *tmp;
p = start;
while(p → next != Null)
{
    If(p → next → info == data)
    {
        tmp = p → next;
        p → next = NULL;
        free(tmp);
    }
    p = p → next
}
    
```

I. while

Delete - 4 & 6

Deletion of Last Node



```

struct node *p;
struct node *tmp;
p = start;
while(p->next != Null)
{
    If(p->next->info == data)
    {
        tmp = p->next;
        p->next = NULL;
        free(tmp);
    }
    p = p->next
}
    
```

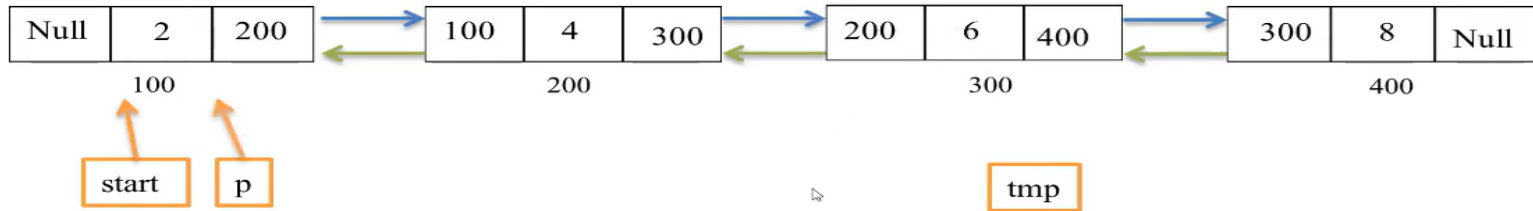
$p = 100$

I. while (200 != null) ✓
 if (4 == data) 4
 $p = 200$

$tmp = 300$
 $300 \rightarrow info = 6$
 II. while (300 != null) ✓
 if (6 == 6) ✓
 $tmp = 300$

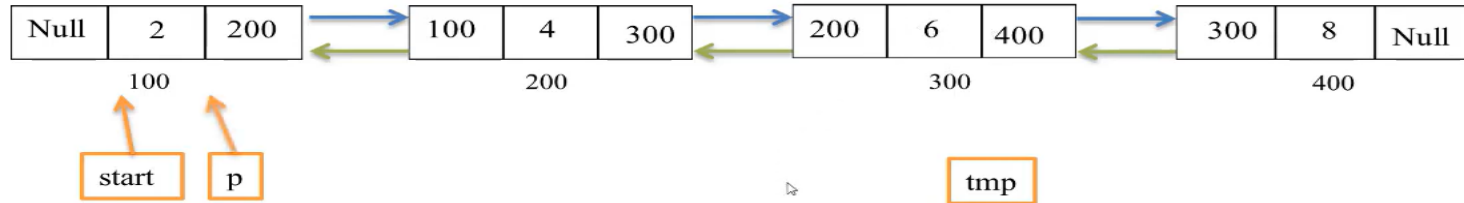
Doubly Linked List

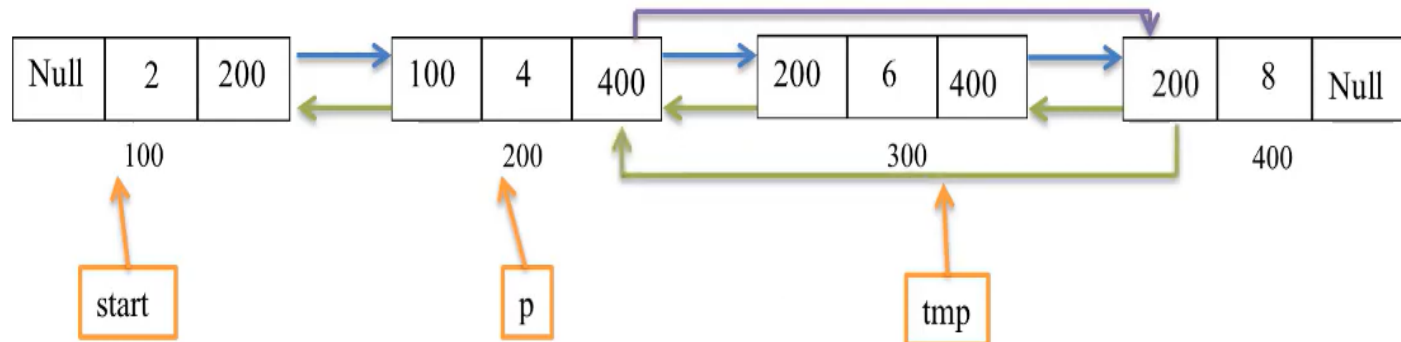
Deletion of nth Node

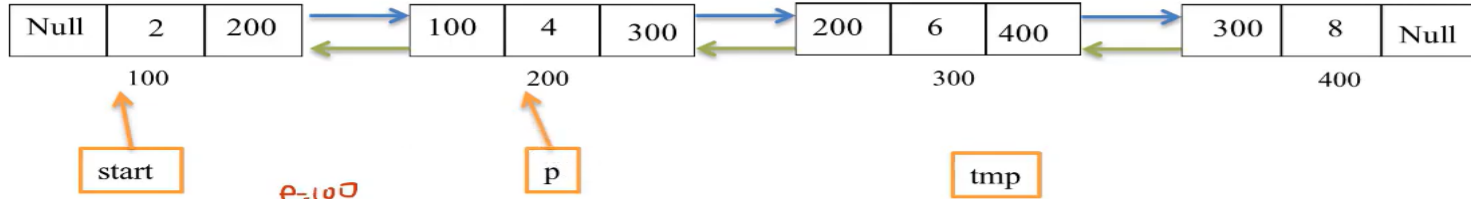


Doubly Linked List

Deletion of nth Node







```
struct node *p, *tmp;
```

```
p = start;
```

```
while(p → next != Null)
```

```
{
```

✓ If(p → next → info == data) ✓

```
{
```

```
tmp = p → next;
```

```
p → next = tmp → next;
```

```
tmp → next → prev = tmp → prev;
```

```
free(tmp);
```

```
}
```

✓ p = p → next ✓

```
}
```

I. while(200 != null) ✓

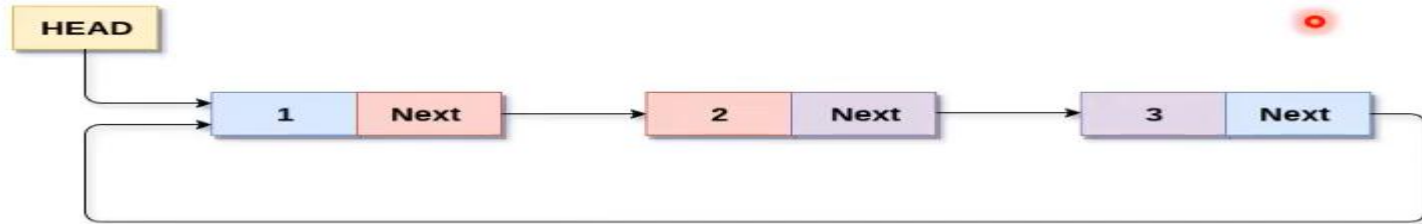
if(4 == 6) ✓

✓ p = 200

II. while(

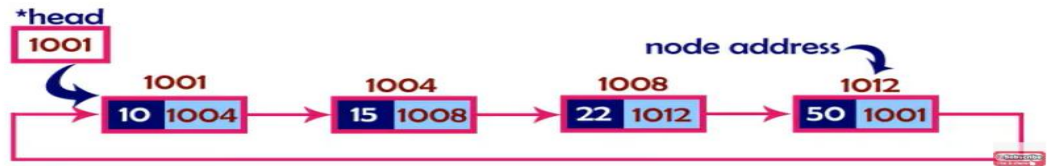
Circular Linked List

The circular linked list is the variation of a linked list where the first node points to the last node, and the last node points to the first node.



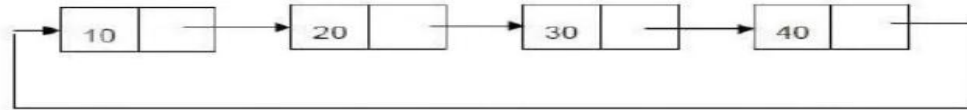
What is Circular Linked List?

- A circular linked list is a sequence of elements in which every element has a link to its next element in the sequence and **the last element has a link to the first element**.
- Both singly and doubly linked list can be made into a circular linked list.
- Circular linked list can be used to help traverse the same list again and again if needed.
- That means circular linked list is similar to the single linked list except that the last node points to the first node in the list

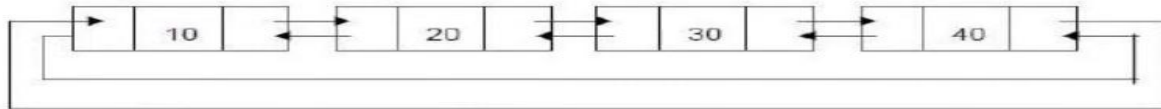


Types of Circular Linked List

1. **Singly:** The last node points to the first node and there is only link between the nodes of linked list.



2. **Doubly:** The last node points to the first node and there are two links between the nodes of linked list.



Operations on Circular Linked List

SN	Operation	Description
1	Insertion at beginning	Adding the node into the linked list at beginning.
2	Insertion at end	Adding the node into the linked list to the end.
3	Insertion after specified node	Adding the node into the linked list after the specified node.
4	Deletion at beginning	Removing the node from beginning of the list
5	Deletion at the end	Removing the node from end of the list.
6	Deletion of the node having given data	Removing the node which is present just after the node containing the given data.
7	Searching	Comparing each node data with the item to be searched and return the location of the item in the list if the item found else return null.
8	Traversing	Visiting each node of the list at least once in order to perform some specific operation like searching, sorting, display, etc.

Declaration of CLL Node

- In order to implement the circular linked list, we maintain an external pointer “**last**” that points to the last node in the circular linked list.
- Hence **last**->**next** will point to the first node in the linked list.
- By doing this we ensure that when we insert a new node at the beginning or at the end of the list, we need not traverse the entire list.

```
struct Node
{
    int data;
    struct Node *next;
};
```

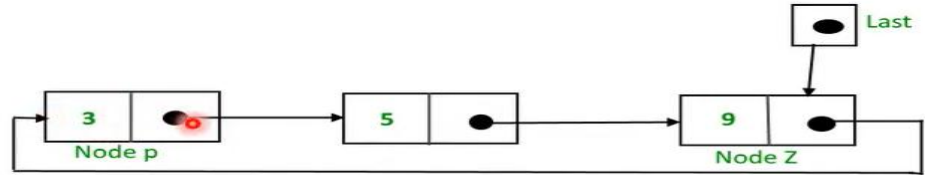
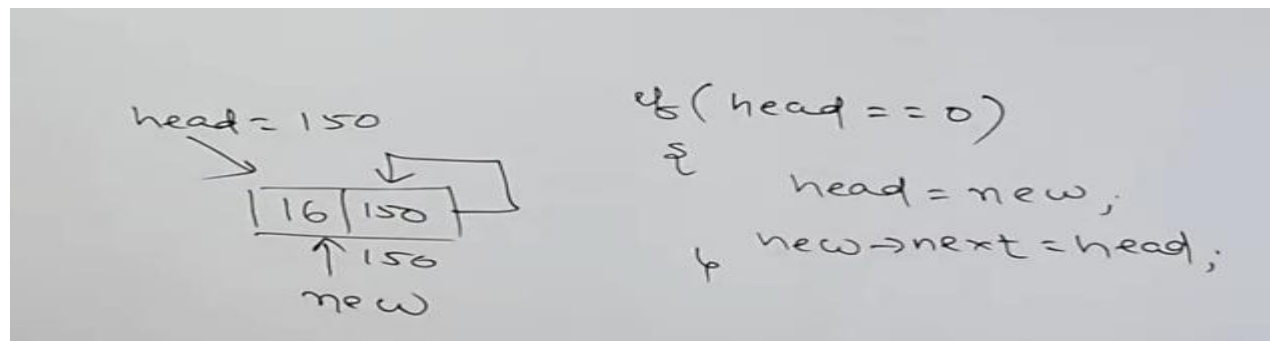
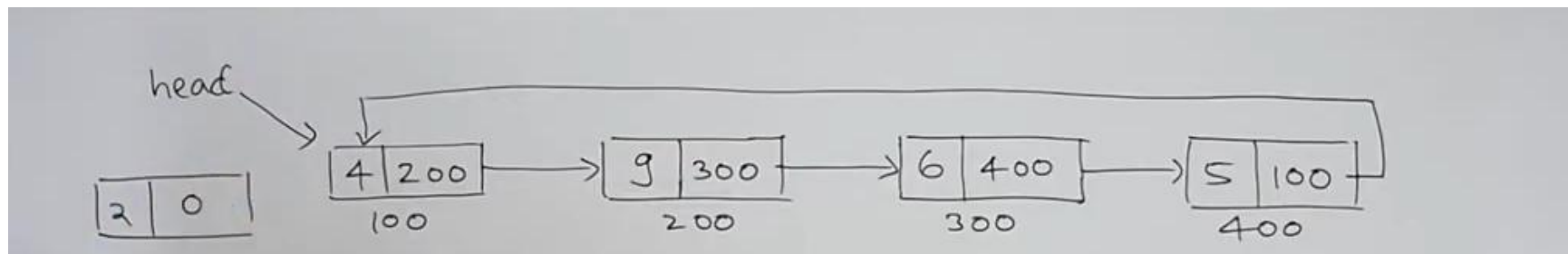


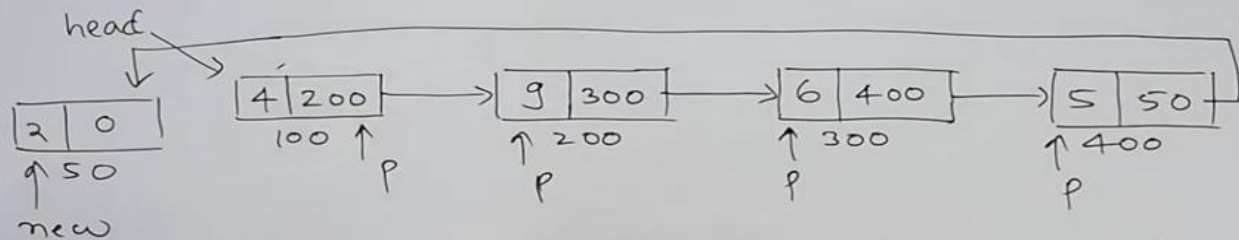
Table Representation:

Add	Data	Link
①	A	2
②	B	3
③	C	4
④	D	1

The table represents a circular linked list where:

- Node at address ① (A) points to node at address ② (B).
- Node at address ② (B) points to node at address ③ (C).
- Node at address ③ (C) points to node at address ④ (D).
- Node at address ④ (D) points back to node at address ① (A), forming a circular structure.

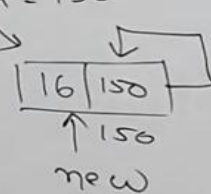




```

struct node * P = head; head = 150
while (P->next != head)
{
    P = P->next;
}
P->next = new;

```



```

if (head == 0)
{
    head = new;
    new->next = head;
}

```

Representation of Circular Linked List

```
struct Node {  
    int data;  
    struct Node * next;  
};
```




```
/* Initialize nodes */
struct node *last;
struct node *one = NULL;
struct node *two = NULL;
struct node *three = NULL;

/* Allocate memory */
one = malloc(sizeof(struct node));
two = malloc(sizeof(struct node));
three = malloc(sizeof(struct node));

/* Assign data values */
one->data = 1;
two->data = 2;
three->data = 3;

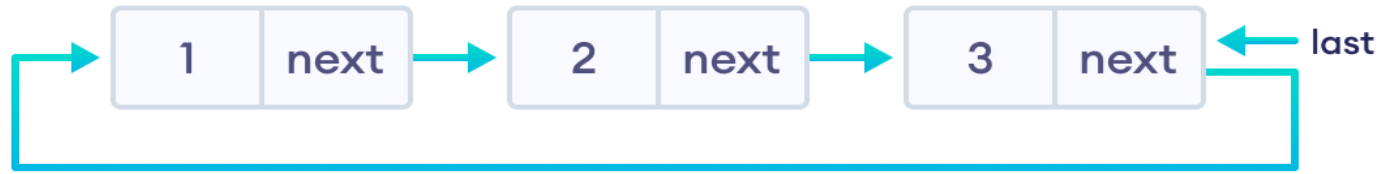
/* Connect nodes */
one->next = two;
two->next = three;
three->next = one;

/* Save address of third node in last */
last = three;
```

Insertion on a Circular Linked List

1. Insertion at the beginning
2. Insertion in-between nodes
3. Insertion at the end

Suppose we have a circular linked list with elements 1, 2, and 3.



Let's add a node with value 6 at different positions of the circular linked list we made above. The first step is to create a new node.

- allocate memory for `newNode`
- assign the data to `newNode`

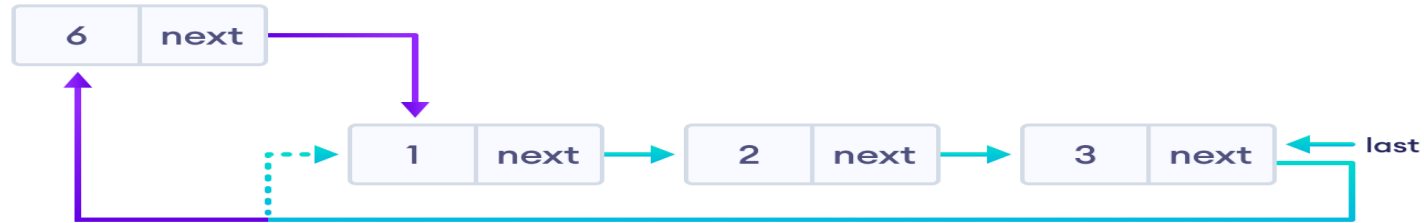


Insertion at the beginning

To insert a new node at the beginning of a circular linked list, we first create the **new node** and allocate memory for it. If the list is empty (indicated by the last pointer being **NULL**), we make the **new node** point to itself. If the list already contains nodes then we set the **new node's** next pointer to point to the **current head** of the list (which is **last->next**), and then update the last node's next pointer to point to the **new node**. This maintains the circular structure of the list.

1. Insertion at the Beginning

- store the address of the current first node in the `newNode` (i.e. pointing the `newNode` to the current first node)
- point the last node to `newNode` (i.e making `newNode` as head)

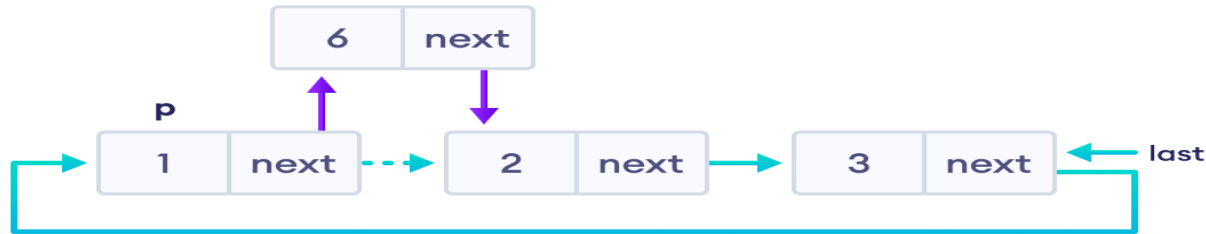


Insertion in between two nodes

2. Insertion in between two nodes

Let's insert newNode after the first node.

- travel to the node given (let this node be p)
- point the `next` of `newNode` to the node next to p
- store the address of `newNode` at `next` of p



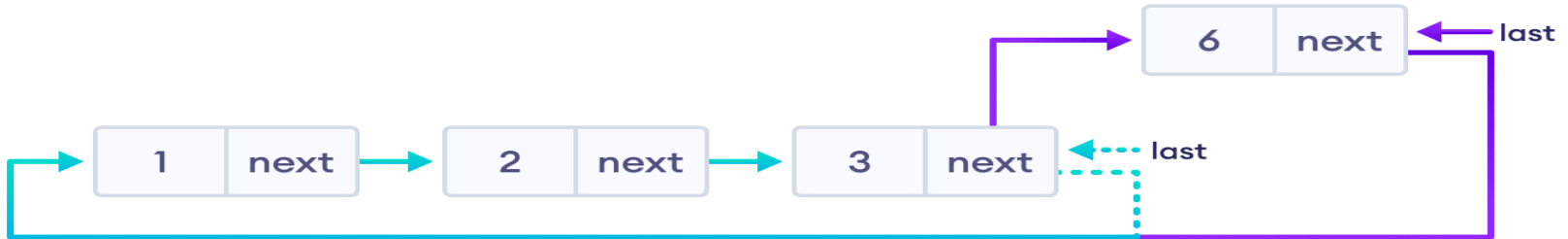
3. Insertion at the end

A circular linked list is a data structure where each node points to the next, and the last node connects back to the first, creating a loop. **Insertion at the end in circular linked list** is an important operation.

Understanding how to perform this insertion is essential for effectively manage and use circular linked lists in various applications.

Insertion at the end

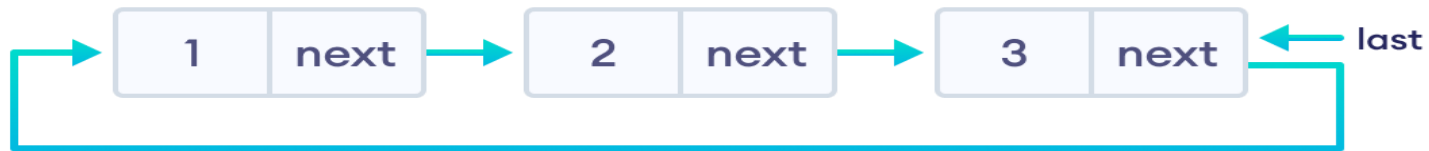
- store the address of the head node to `next` of `newNode` (making `newNode` the last node)
- point the current last node to `newNode`
- make `newNode` as the last node



Deletion on a Circular Linked List

1. Deletion from the beginning of the circular linked list

To delete the first node of a circular linked list, we first check if the list is empty. If it is then we print a message and return **NULL**. If the list contains only one node (the **head** is the same as the **last**) then we delete that node and set the **last** pointer to **NULL**. If there are multiple nodes then we update the **last->next** pointer to skip the **head** node and effectively removing it from the list. We then delete the **head** node to free the allocated memory. Finally, we return the updated **last** pointer, which still points to the **last** node in the list.



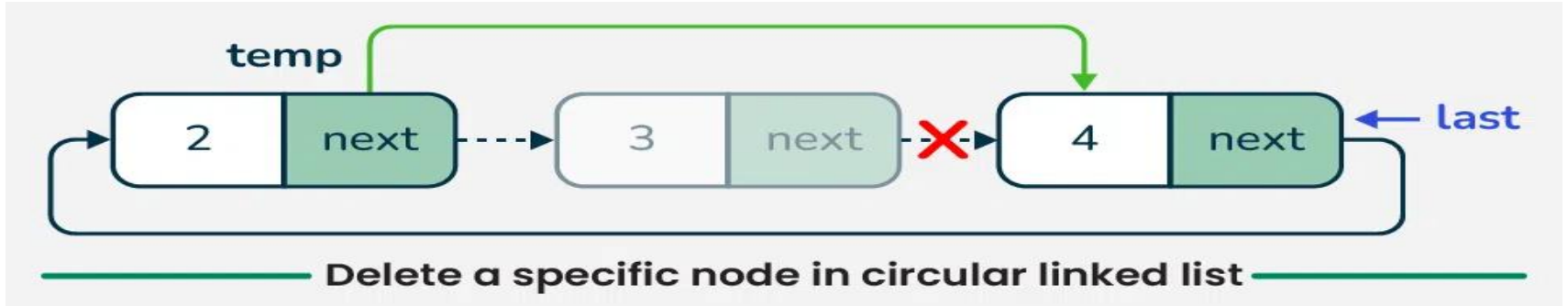
Deletion on a Circular Linked List

Step-by-step approach:

- Check if List is Empty:
 - If **last** is **nullptr**, print “**List is empty**” and return **nullptr**.
- Get Head Node:
 - Set **head** to **last->next**.
- Check for Single Node:
 - If **head** equals **last**, delete **head** and set **last** to **nullptr**.
- Handle Multiple Nodes:
 - Update **last->next** to point to **head->next**.
 - Delete **head**.
- Return Updated **last**

Deletion at specific position in circular linked list

To delete a specific node from a circular linked list, we first check if the list is empty. If it is then we print a message and return **nullptr**. If the list contains only one node and it matches the **key** then we delete that node and set **last** to **nullptr**. If the node to be deleted is the first node then we update the **next** pointer of the **last** node to skip the **head** node and delete the **head**. For other nodes, we traverse the list using two pointers: **curr** (to find the node) and **prev** (to keep track of the previous node). If we find the node with the matching key then we update the next pointer of **prev** to skip the **curr** node and delete it. If the node is found and it is the last node, we update the **last** pointer accordingly. If the node is not found then do nothing and **tail** or **last** as it is. Finally, we return the updated **last** pointer.

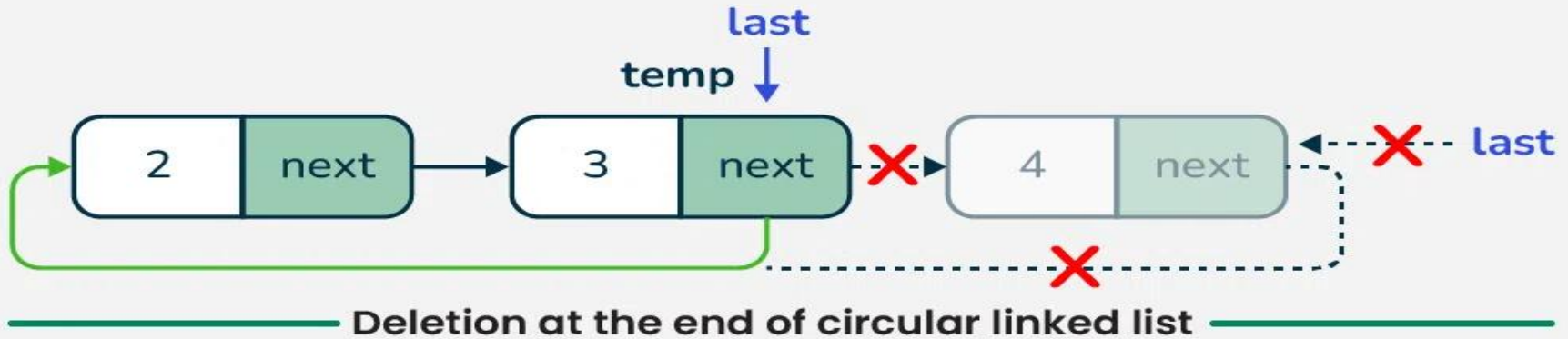


Step-by-step approach:

- Check if List is Empty (last is nullptr) then print “**List is empty, nothing to delete**” and return **nullptr**.
- Set **curr** to **last->next** (head) and **prev** to **last**.
- Check for single node, if node’s data matches **key**, delete it and set **last** to **nullptr**.
- Check for first node to delete, if head node’s data matches **key**, update **last->next** and delete the **head**.
- Loop through the list to find the node with the specified **key**.
 - If node found then update **prev->next** to skip the deleted node. If the deleted node is **last**, update **last** to **prev**.
 - Otherwise, print “**Node with data [key] not found.**”
- Return the modified **last**.

Deletion at the end of Circular linked list

To delete the last node in a circular linked list, we first check if the list is empty. If it is, we print a message and return **nullptr**. If the list contains only one node (where the **head** is the same as the **last**), we delete that node and set **last** to **nullptr**. For lists with multiple nodes, we need to traverse the list to find the **second last node**. We do this by starting from the **head** and moving through the list until we reach the node whose next pointer points to **last**. Once we find the **second last** node then we update its next pointer to point back to the **head**, this effectively removing the last node from the list. We then delete the last node to free up memory and return the updated **last** pointer, which now points to the last node.



Step-by-step approach:

- Check if List is Empty (**last** is **nullptr**) then print “**List is empty, nothing to delete**” and return **nullptr**.
- Get **head** node by Setting **head** to **last->next**.
- Check for single node, if **head** equals **last**, delete **last** and set **last** to **nullptr**.
- Find second last node by traversing the list until **curr->next** equals **last**.
- Update Pointer by setting **curr->next** to point to **head**.
- Delete **last** and update **last** to **curr**.
- Return the updated **last**.

QUEUE

Queue is a linear collection of data item that follows FIFO (First-in-First-out) where insertion and deletion take place on opposite end. The insertion end is called REAR and deletion end is called Front.

Note:

Enqueue

→ Insertion of item

Dequeue

→ Deletion of item

Queue

→ Array

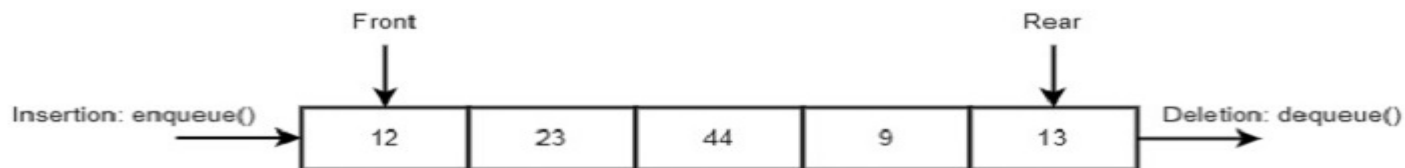
→ Linked List



A real-world example of queue can be a single-lane one-way road, where the vehicle enters first, exits first. More real-world examples can be seen as queues at the ticket windows and bus-stops.

Representation of Queues

Similar to the stack ADT, a queue ADT can also be implemented using arrays, linked lists, or pointers.



Queue: FIFO Operation

Queue Insertion Operation: Enqueue()

The enqueue() is a data manipulation operation that is used to insert elements into the queue. The following algorithm describes the enqueue() operation in a simpler way.

Algorithm

1. START
2. Check if the queue is full.
3. If the queue is full, produce overflow error and exit.
4. If the queue is not full, increment rear pointer to point the next empty space.
5. Add data element to the queue location, where the rear is pointing.
6. return success.
7. END

Algorithm (Insertion in Queue)

Step 1: Begin

Step 2: If $Rear = N - 1$ then

 Print "overflow & exit"

Step 3: Input new item

Step 4: $Rear \leftarrow Rear + 1$

Step 5: $Queue[Rear] \leftarrow item$

Step 6: Exit

Queue Deletion Operation: dequeue()

The dequeue() is a data manipulation operation that is used to remove elements from the stack. The following algorithm describes the dequeue() operation in a simpler way.

Algorithm

1. START
2. Check if the queue is empty.
3. If the queue is empty, produce underflow error and exit.
4. If the queue is not empty, access the data where front is pointing.
5. Increment front pointer to point to the next available data element.
6. Return success.
7. END

Algorithm (Deletion from Queue)

Step 1: Begin

Step 2: If `Front = -1` then

 Print "underflow & exit"

Step 3: Set `item ← Queue[Front]`

Step 4: `Front ← Front + 1`

Step 5: Print "item deleted"

Step 6: Exit